Magnus Tomter Ouren & Jakob Elias Relling

# Attention-Aware Feedback for Novice Debuggers

A Systematic Review and Preparatory Design for Attention-Based Feedback during debugging

IT3915 – Master in Informatics, Preparatory Project
Semester: Autumn 2025
Supervisor: Khsithj Sharma
Co-supervisor: Anahita Golrang

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Computer Science

NTNU

# ABSTRACT

This preparatory project examines how eye tracking can support the design of attention-aware feedback for novice programmers during debugging. A Systematic Literature Review of 48 peer-reviewed studies from 2012 to 2025 was conducted to understand how visual attention, cognitive load, and performance have been studied in programming tasks.

The review shows that most studies focus on code comprehension with short Java or C-like snippets and rely heavily on fixation-based metrics. Debugging tasks are uncommon, and almost none use gaze data to trigger feedback during the task. Key challenges include limited ecological validity, inconsistent performance measures, and little use of real-time or physiological indicators. The findings point to clear opportunities for research on attention-informed support for novices.

Based on these gaps, the project outlines the design for the upcoming master's study, which will compare proactive and reactive feedback conditions using a prototype system that detects attention patterns during debugging. This work provides the foundation for developing feedback mechanisms that can help novices debug more effectively.

# CONTENTS

**Abstract** **i**

**Contents** **iii**

**List of Figures** **iii**

**List of Tables** **iv**

**1 Introduction** **2**
1.1 Motivation . . . . . 2
1.2 Why Debugging and Eye Tracking . . . . . 2
1.3 Research Gap and Relevance . . . . . 3
1.4 Lack of Attention-Based Feedback Approaches . . . . . 4
1.5 Timing of Feedback Activation . . . . . 4
1.6 Research Objectives and Research Questions . . . . . 5
1.7 Structure of the Report . . . . . 5

**2 Systematic Literature Review** **7**
2.1 Introduction . . . . . 7
2.1.1 Inquiry Question Guiding the Review . . . . . 7
2.2 Methodology . . . . . 9
2.2.1 Preliminary Phase . . . . . 9
2.2.2 Search Strategy . . . . . 10
2.2.3 Inclusion and Exclusion Criteria . . . . . 12
2.2.4 Screening and Selection Process . . . . . 13
2.2.5 Data Extraction and Coding . . . . . 14
2.2.6 Thematic Synthesis . . . . . 19
2.3 Descriptive Findings . . . . . 21
2.3.1 Study Characteristics . . . . . 21
2.3.2 Population Characteristics . . . . . 23
2.3.3 Analytical and Measurement Approaches . . . . . 24
2.3.4 Summary . . . . . 26
2.4 Thematic Synthesis . . . . . 28
2.4.1 State of the Art . . . . . 28
2.4.2 Methodological Challenges . . . . . 33
2.4.3 Opportunities . . . . . 37
2.5 Summary . . . . . 42

2.6 Limitations . . . . . 43

**3 Design and Future Plan 45**
3.1 Introduction . . . . . 45
3.2 Research Design . . . . . 45
3.2.1 Introduction . . . . . 45
3.2.2 Participants and context . . . . . 46
3.2.3 Experimental design and feedback conditions . . . . . 46
3.2.4 Procedure . . . . . 46
3.2.5 Measures and data collection . . . . . 47
3.2.6 Ethical considerations . . . . . 48
3.3 Technical Design . . . . . 48
3.3.1 Introduction . . . . . 48
3.3.2 System Overview . . . . . 48
3.3.3 Data Inputs and Context . . . . . 48
3.3.4 Feedback Logic . . . . . 49
3.3.5 Feedback Generation . . . . . 49
3.3.6 User Interaction w/ Feedback . . . . . 50
3.3.7 Data Storage . . . . . 51
3.3.8 Summary . . . . . 51
3.4 Future Plan . . . . . 52

**References 55**

**Appendices: 65**

**A - Overview of Coded Studies 66**

**B - Figma Prototype 72**

# LIST OF FIGURES

2.1.1 Overview of the SLR process followed in this review. . . . . 8
2.2.1 Full Boolean search query. . . . . 12
2.2.2 PRISMA-style flow of the study identification, screening, and selection process. . . . . 15
2.3.1 Overview of study characteristics across the 48 included studies . . 22
2.3.2 Overview of population characteristics across the 48 included studies. 24
2.3.3 Overview of analytical and measurement approaches across the 48 included studies . . . . . 27

3.2.1 Research design with counterbalanced within-subject feedback conditions. . . . . 47
3.3.1 Narrow system flow with proactive (left) and reactive (right) feedback paths. . . . . 49
3.3.2 Interaction flow when feedback is generated during a task. Users may accept or reject feedback, temporarily disable it, or mute it for the remainder of the current task. . . . . 50
3.3.3 Notification informing the user that feedback is available . . . . . 51
3.3.4 Prototype showing how accepted feedback is presented to the user in VS Code . . . . . 52
B.1 Example of UI after feedback is generated . . . . . 73
B.2 Example of UI after feedback is presented . . . . . 73
B.3 Example of UI after feedback is rejected . . . . . 74
B.4 Example of UI after feedback is disabled . . . . . 74

# LIST OF TABLES

2.2.1 Search concepts and corresponding keywords. . . . . 11
2.2.2 Selection Criteria . . . . . 14
2.2.3 SLR coding table . . . . . 17
2.2.4 Coding scheme for evaluation of performance metrics. . . . . 18
2.2.5 Categories of gaze and physiological metrics used in the reviewed studies. . . . . 18
2.2.6 Categories of analytical and statistical methods used in the reviewed studies. . . . . 19
2.2.7 Example of a coded study record. . . . . 20
2.4.1 Overview of themes and corresponding papers included in the thematic synthesis. . . . . 29
2.4.2 Overview of challenges and corresponding papers included in the thematic synthesis. . . . . 34
2.4.3 Overview of opportunities and corresponding papers included in the thematic synthesis. . . . . 38

3.4.1 Tentative timeline and stages for the master's thesis project. . . . . 53
A.1 Coded overview of all included studies. . . . . 67

CHAPTER

# ONE

# INTRODUCTION

## 1.1 Motivation

Debugging is among the most cognitively demanding aspects of learning to program, requiring learners to maintain focus, test hypotheses, and reason about program behaviour under uncertainty. Many novices struggle to identify where to begin or how to proceed when their code fails, often relying on trial-and-error approaches rather than systematic strategies. As a result, feedback plays a crucial role in guiding attention, reducing frustration, and fostering deeper understanding during the debugging process.

Recent advances in eye-tracking and learning analytics make it possible to observe how programmers allocate visual attention while solving problems. These insights can inform how and when feedback should be delivered to be most effective. While most instructional tools today provide reactive feedback, there is growing interest in exploring proactive feedback that anticipates confusion or cognitive overload based on real-time behavioural or attentional signals. Such adaptive feedback could help novices maintain focus and develop more effective debugging strategies.

This preparatory study aims to identify methodological trends, research gaps, and opportunities related to how attention and feedback have been addressed in prior eye-tracking research on debugging. The findings will serve as the foundation for the coming master's thesis, which will investigate how proactive and reactive feedback mechanisms can be designed and evaluated to better support novices during debugging.

## 1.2 Why Debugging and Eye Tracking

Debugging requires learners to read code carefully, form and test hypotheses, and reason about program behaviour under uncertainty. Many novices find debugging harder than writing code, often relying on trial and error rather than systematic strategies [1, 2]. A key challenge is that beginners frequently lack a clear mental model of runtime behaviour, which makes it difficult to identify where faults originate [3].

Research shows that novices tend to approach debugging without structure, for

example by jumping between code fragments or making unsystematic edits, while more experienced programmers trace control flow or reason backward from faulty output [4, 5]. Eye-tracking studies also indicate that successful debuggers maintain more coherent reading patterns, while less successful ones exhibit scattered gaze behaviour [5].

Despite its importance, debugging is rarely taught explicitly. Courses often prioritise writing new code, leaving students to develop debugging strategies independently, even though productive debugging deepens understanding of program logic and behaviour [6]. Automated tools can assist with syntax or simple issues, but deeper bugs still require human reasoning and attention [7].

Because debugging depends heavily on managing visual attention, such as tracing control flow, inspecting variables, and shifting focus between representations, eye-tracking provides an appropriate method for studying it. Gaze data reveal where programmers look, for how long, and how they transition between code elements, giving insight into underlying reasoning processes.

Traditional methods for studying cognitive load, for example, questionnaires or think-aloud protocols, offer valuable information but rely on self-reporting and can interfere with natural problem solving [8, 9]. Eye-tracking offers a non-intrusive alternative that captures attention patterns, cognitive load indicators like pupil dilation, and strategy shifts in real time [10, 11]. This makes it well-suited for investigating how novices navigate debugging tasks and where they encounter difficulties.

## 1.3 Research Gap and Relevance

The systematic literature review in Chapter 2 shows that eye tracking has been used extensively to study code comprehension, problem solving, and expertise differences in programming. Most of the 48 included studies focus on short comprehension tasks in Java or C-like languages, with individual students working on carefully controlled snippets. Eye tracking is typically employed as a diagnostic tool to characterize visual attention, cognitive load, or strategy use, and effects are mainly reported in terms of fixation metrics, task time, and correctness.

The review also highlights several gaps that motivate the present work. Debugging is clearly underrepresented. Only a small subset of studies includes debugging or bug-finding tasks, and even fewer concentrate on novice programmers working with realistic, iterative debugging activities. As a result, we know comparatively little about how novices allocate attention, experience cognitive load, and progress through the different phases of debugging, beyond what can be inferred from comprehension-oriented studies.

Few of these studies use eye tracking or related measurements to drive feedback during the task itself. In most cases, gaze and performance data are analysed after the fact to explain behaviour, rather than being used to trigger instructional support. Only a handful of papers in the corpus implement any form of feedback, and only one combines feedback with explicit debugging tasks. This means there is little empirical evidence on how attention-informed feedback should be designed, when it should be activated, and how it affects learning outcomes in debugging.

Although several studies point to the importance of cognitive load and carry-

forward effects across task phases, there is limited work that connects these insights to concrete, adaptive support mechanisms. Existing research rarely distinguishes systematically between proactive feedback (anticipating difficulties based on real-time indicators) and reactive feedback (responding after errors or failures), and almost none evaluate these approaches in novice debugging contexts.

These gaps make debugging with novice programmers, attention-aware measurements, and feedback-based interventions a relevant and timely area for further research. By designing and evaluating feedback mechanisms that are informed by visual attention and cognitive load during debugging, the coming master's thesis aims to extend the current state of the art and provide actionable implications for programming education and learning technologies.

## 1.4 Lack of Attention-Based Feedback Approaches

Existing literature reveals a clear scarcity of attention-aware feedback during debugging. Most eye-tracking studies in programming use gaze data purely for diagnosis after tasks are completed, rather than to guide or support learners in real time. Eye movement metrics are typically collected and analyzed post-hoc to explain what the student did, with almost no studies leveraging these measurements to trigger immediate feedback while the debugging is in progress. In fact, out of the 48 studies reviewed in our systematic analysis, only a single study attempted any form of attention-driven feedback in a debugging context. This lone example underscores how uncommon it is for current research to integrate visual attention signals into live interventions for novice debuggers.

## 1.5 Timing of Feedback Activation

Prior research highlights feedback as one of the most influential factors for learning, with large effects on student achievement [12]. A central question is therefore when feedback should be activated. In programming education, two common activation strategies are proactive and reactive feedback.

Proactive feedback is initiated by the system. The tutor, or in our case LLM generated feedback, will monitor the learner's behaviour and provides hints or guidance before the learner explicitly asks for help. This can reduce unproductive struggle and support students who don't utilise available help. For example, [13] uses predictive modeling in block-based programming to identify early signs of difficulty and proactively intervene when learners are likely to get stuck. Earlier work with intelligent tutors shows similar patterns, with immediate, controlled feedback enabling students to complete tasks more efficiently. Even maintaining similar learning outcomes to other conditions [14]. These findings suggest that timely proactive feedback can keep novices on track without reducing the final achievement.

Reactive feedback is activated after the learner has taken an action, for example, when they make an error or when they ask for help. This timing encourages productive struggle because novices are given a chance to try out their own ideas before receiving guidance. It can also support the development of debugging and

self-monitoring skills, since students must identify and interpret their own mistakes before the system responds. The usefulness of this type of feedback depends heavily on how learners seek help. Earlier studies show that many novices either avoid asking for assistance when they need it or use hints in ineffective ways, and this reduces the overall value of systems that rely only on reactive feedback [15].

## 1.6 Research Objectives and Research Questions

While the previous sections outlined why debugging is a cognitively demanding activity, how eye tracking can reveal difficulty, and why attention-based feedback remains underexplored, this section translates those insights into a clear and actionable research agenda for the master's thesis. The objective defines the conceptual focus of the study, whereas the research questions operationalise it into measurable components that will guide the experimental design, data collection, and analysis strategy. Together, they ensure that the planned work meaningfully extends prior literature while directly addressing the limitations highlighted in the systematic review.

**Research Objective:** *To design and evaluate an attention-aware feedback system that supports novice programmers during debugging by comparing how proactive and reactive feedback influence their performance, cognitive load, and visual attention.*

**Research Questions:**

**RQ1** How does proactive and reactive feedback affect debugging performance for novice programmers?

**RQ2** Which real-time eye-tracking indicators can signal moments of difficulty for novice programmers during debugging?

**RQ3** How do novice programmers perceive the usefulness, timing, and intrusiveness of proactive versus reactive feedback during debugging?

**RQ4** How do cognitive load and gaze behaviour differ across the control, proactive feedback, and reactive feedback debugging conditions?

## 1.7 Structure of the Report

This preparatory project is structured into four chapters, each serving a specific purpose in establishing the foundation for the forthcoming master's thesis.

Chapter 1 introduces the topic, outlines the motivation for investigating debugging with eye tracking, and presents the research gap identified in the field. It further specifies the research objectives and questions that guide the planned study.

Chapter 2 presents the Systematic Literature Review (SLR). This chapter describes the methodological procedures used in the review, including search strategy, screening, coding, and synthesis. It then provides descriptive findings and a

thematic integration of prior work, which together form the empirical basis for the design of the forthcoming study.

Chapter 3 outlines the research design for the planned experiment. It describes the participant context, the within-subject design comparing proactive and reactive feedback, the experimental procedure, the selected measures, and the ethical considerations relevant to data collection. It also presents the technical design of the prototype system that will support the study. Specifying the system architecture, data inputs, feedback triggering mechanisms, interaction flow, and data storage strategy, linking the conceptual research design to the practical implementation required for the experiment.

CHAPTER

# TWO

# SYSTEMATIC LITERATURE REVIEW

## 2.1 Introduction

This chapter presents the Systematic Literature Review (SLR) conducted to map how eye-tracking has been used in studies involving debugging tasks among novice programmers. The review follows established SLR guidelines [16] to ensure a transparent and reproducible process. It is guided by a central inquiry question, presented in the next subsection (Section 2.1.1), which defines the focus and analytical scope of the review.

The chapter is structured as follows. Section 2.2 describes the methodological design, including the preliminary phase, search strategy, inclusion and exclusion criteria, and data extraction procedure. Section 2.3 presents the descriptive findings of the reviewed studies, summarizing their main characteristics in terms of task design, participant populations, and analytical approaches. Section 2.4 provides the thematic synthesis, highlighting conceptual trends, methodological challenges, and research opportunities. Together, these sections offer a comprehensive overview of how visual attention has been investigated in programming and debugging contexts and form the empirical basis for the subsequent research design. An overview of the overall process followed in this review is illustrated in Figure 2.1.1.

### 2.1.1 Inquiry Question Guiding the Review

The purpose of this systematic literature review (SLR) is to establish a structured overview of how eye-tracking has been applied in studies involving debugging tasks among novice programmers. Debugging is one of the most cognitively demanding aspects of learning to program, requiring sustained attention, hypothesis testing, and reasoning about program behaviour. Eye-tracking provides a powerful method for capturing these attentional processes in real time, making it well-suited for understanding how novices approach and navigate debugging activities.

Although prior research in computing education has examined code comprehension, problem solving, and expertise differences through eye-tracking, relatively few studies focus specifically on debugging as an iterative and error-driven task. Existing work varies widely in experimental design, task types, metrics, and analytical frameworks, making it difficult to form a consolidated understanding of

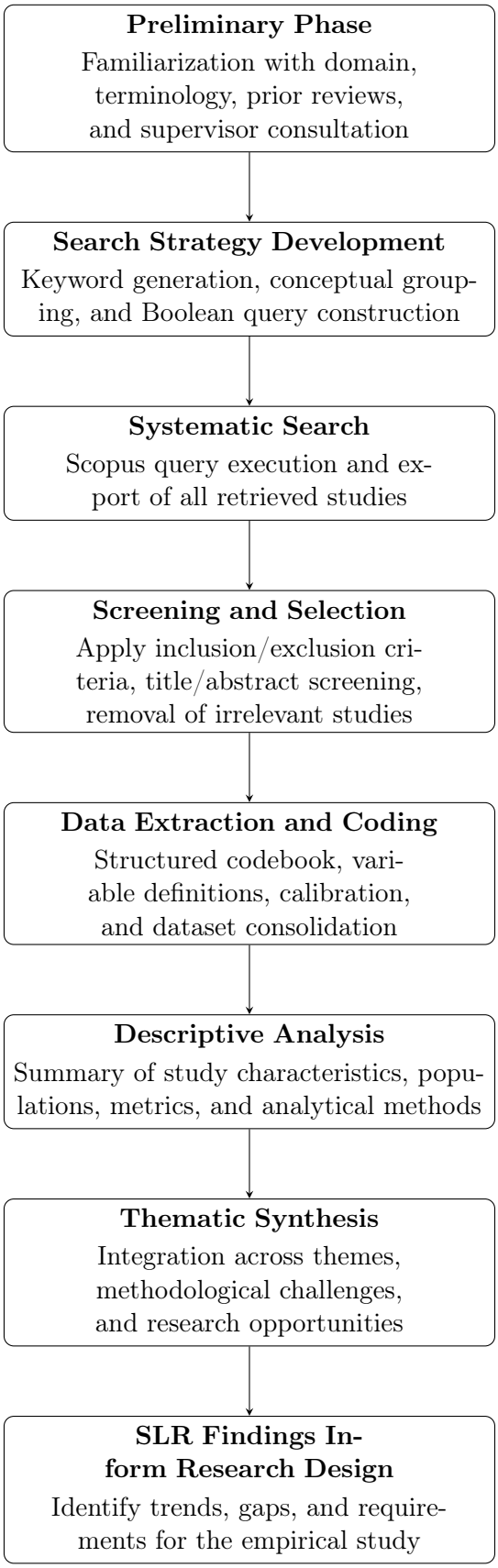



**Figure 2.1.1:** Overview of the SLR process followed in this review.

how visual attention in debugging has been studied to date.

To address this, the present review is guided by the following inquiry question:

> *How has eye-tracking been applied to investigate novice programmers' attention and cognitive processes during debugging tasks, and what methodological trends and research gaps can be identified across existing studies?*

This inquiry directs the review in two ways. First, it seeks to characterize how prior studies have operationalized and measured visual attention, cognitive load, and performance within debugging-related contexts. Second, it aims to identify methodological patterns, limitations, and underexplored opportunities that can inform the design of the coming master's thesis's empirical work.

#### 2.1.1.1 Sub-Inquiry Questions

To operationalize the main inquiry question, it is subdivided into the following Sub-Inquiry Questions (SIQs), each targeting a specific dimension of the reviewed literature:

**SIQ1** How are debugging-related tasks operationalized across studies (e.g., task type, programming language, and experimental setup)?

**SIQ2** What populations do these studies involve, and how might sample characteristics influence insights about novice debugging behaviour?

**SIQ3** What metrics and instrumentation do studies use to capture visual attention and cognitive effort during debugging-related tasks?

**SIQ4** How do studies measure debugging performance, and which types of performance outcomes are reported?

**SIQ5** What analytical methods are used to interpret visual attention and performance data in debugging-related research?

**SIQ6** To what extent are feedback mechanisms included in prior research, and how are feedback type, delivery, and triggers described?

**SIQ7** How do the studies' stated research questions and objectives align with their operationalization of debugging, attention, and measurement?

**SIQ8** What methodological limitations and future work are identified across the studies, and what do these suggest about gaps in the current research landscape?

## 2.2 Methodology

### 2.2.1 Preliminary Phase

Before initiating the formal search and selection procedures, a preliminary phase was conducted to refine the understanding of the research domain and to ensure

that the review protocol was well aligned with established guidelines. According to [16], an SLR should begin with a planning stage in which researchers become familiar with the topic, definition of key concepts, and verify the feasibility of the study design.

During this pre-phase, existing reviews on eye-tracking and research related to programming were reviewed to identify recurring themes, conceptual frameworks, and methodological approaches. This preliminary exploration helped clarify the terminology used in the field and informed the later construction of keyword groups.

Furthermore, consultations with the supervisor, an expert in learning Technology and analytics, was held to validate the understanding of the domain and to ensure that the planned focus on debugging and novice programmers was both relevant and feasible. These discussions contributed to a clearer definition of the scope of the review, refinement of the research questions, and alignment of the study methodology with established SLR procedures [16].

### 2.2.2 Search Strategy

A systematic and reproducible search strategy was adopted to identify as many relevant studies as possible while minimizing researcher bias. The entire search process, including keyword generation, database selection, and query construction, was documented to ensure transparency and reproducibility [16]. The process was continuously guided by the supervisor, who ensured that both the scope and terminology of the search remained consistent with the research objectives and current developments in the field.

#### 2.2.2.1 Keywords

The identification and refinement of relevant keywords was a key, iterative step in developing a comprehensive and unbiased search strategy. In line with the methodological guidance of [16], a structured brainstorming process was used to construct search terms that reflected not only the inquiry question but also the motivation of this project: understanding how novices debug and how attention-aware feedback could support them. To capture these core concepts, the final keyword set was organized into three groups: (i) debugging and error correction, (ii) eye-tracking and visual attention, and (iii) novice programmers.

This grouping ensured that each component of the search query aligned with a fundamental part of the project's purpose. Terms related to debugging were included because the project focuses on supporting novices during debugging, one of the most cognitively demanding programming activities described in the motivation (Section 1.1). Eye-tracking and attention-related terms were required to identify studies that reveal how visual attention unfolds during problem solving. Finally, terms describing novice programmers ensured that the search retrieved literature relevant to the target population most in need of such support.

The resulting Boolean search query required at least one term from each conceptual group, helping ensure that identified studies were relevant to both the empirical context (novice debugging) and the overarching motivation of designing feedback informed by attention. Table 2.2.1 outlines the final keywords associated with each group.

| Concept | Keywords |
|---|---|
| Debugging and Error Correction | debug, debugging, program debugging, bug fix, bug fixing, software troubleshooting, error correction, program comprehension, find the bug, error |
| Eye Tracking and Visual Attention | eye track, eyetracking, gaze tracking, visual attention, scanpath analysis, pupil dilation |
| Novice Programmers | novice, beginner, student, programming learner, novice programmer, inexperienced programmer, K-12 |

**Table 2.2.1:** Search concepts and corresponding keywords.

#### 2.2.2.2 Databases

The literature search was conducted systematically using the Scopus database due to its extensive multidisciplinary coverage and robust indexing of peer-reviewed content. While we acknowledge that an exclusive search of a single database may introduce selection bias and potentially omit relevant publications not covered by Scopus, the large number of relevant databases (SpringerLink, Wiley, Assembly of Computer Machineries [ACM] Digital Library, IEEE Xplore, Science Direct), and the large number of documents retrieved ($n = 237$) suggests a sufficiently comprehensive set of literature for the purpose of this review. The search was executed on September 18th, 2025, and all retrieved records were exported to Zotero in .ris format for document and metadata storage, and exported as .csv to Google Sheet for further screening and selection.

#### 2.2.2.3 Search Query

The final search string was developed using Boolean operators and wildcards, following the recommendations of [16] for constructing transparent and reproducible queries. The query was structured in two layers to balance sensitivity and precision.

First, the `ALL()` operator was used to identify papers that contained relevant terms across the full record, including titles, abstracts, keywords, and indexing metadata. This broad scope ensured that potentially relevant studies were not excluded due to variations in terminology or indexing.

Secondly, to increase precision and ensure topical relevance `TITLE-ABS-KEY()` operator was added to limit results to papers in which the core terms appeared specifically in the title, abstract, or author keywords.

Date of extraction from Scopus: 18.09.2025

Executing this query in Scopus produced 237 initial records. These results formed the basis for the next phase of the review, in which inclusion and exclusion criteria were systematically applied to remove irrelevant studies.

```
ALL (
    ( debugging OR "program debugging" OR "bug fix*"
      OR "software troubleshoot*" OR "error correction*" OR
      "program comprehension"
    )
    AND
    ( "eye track*" OR eyetracking OR "gaze track*" OR
      "visual attention" OR "scanpath analysis" OR "pupil dilation"
    )
    AND
    ( novice OR beginner OR student OR "programming learner" OR
      "novice programmer" OR "inexperienced programmer"
    )
)
AND TITLE-ABS-KEY (
    ( "eye track*" OR eyetracking OR "gaze tracking" OR "visual
      attention" OR "scanpath analysis" OR "pupil dilation"
    )
    AND
    ( novice OR beginner OR student OR "programming learner"
      OR "novice programmer" OR "inexperienced programmer"
    )
)
```

**Figure 2.2.1:** Full Boolean search query.

### 2.2.3 Inclusion and Exclusion Criteria

Clearly defined inclusion and exclusion criteria are essential for ensuring a consistent and unbiased selection process. Following the recommendations of [16], these criteria were established prior to screening and guided all subsequent stages.

#### Inclusion Criteria

- **Year: 2011–2025** Studies were limited to studies published from 2011 onwards to ensure that the included research reflects contemporary technological and methodological developments in both eye-tracking and programming education. Since 2010, eye-tracking technology has undergone major advances in sampling accuracy, calibration stability, and analysis software, making it suitable for realistic programming and debugging contexts. Restricting the time frame, therefore, increases comparability and methodological consistency across studies.

- **Language: English** Limited to English-language publications to ensure accurate interpretation without translation bias.

- **Document type: Peer-reviewed, journal articles or conference papers** Limiting the dataset to peer-reviewed publications ensures quality control and excludes non-archival or informal sources.

- **Publication status: Published (final) papers** Prevents version drift and ensures that all included studies represent stable contributions.

- **Scope: Studies addressing code comprehension, debugging, or related programming tasks using eye-tracking data** Includes studies on code comprehension, debugging, or related activities involving gaze-based empirical data.

**Exclusion Criteria**

- **Published before 2011** Older studies often used outdated eye-tracking hardware or analytical frameworks that differ substantially from modern standards.

- **Non-peer-reviewed material** (e.g., preprints, theses, technical reports) Excluded to maintain publication quality.

- **Word count below 5000 words (excluding references)** Very short papers rarely provide sufficient methodological details for reliable data extraction and coding. These details could be task design, instrumentation, preprocessing, and performance measures.

- **No empirical data** The review focuses on primary studies presenting quantitative or qualitative data derived from eye-tracking experiments.

- **Webcam-based eye-tracking setups** These were excluded because of lower sampling rates and calibration accuracy compared to dedicated hardware, which would reduce the validity of gaze metrics.

- **Lack of programming content** Papers that did not include programming-related activities (e.g., reading or writing code, debugging, or code comprehension) were excluded.

### 2.2.4 Screening and Selection Process

The initial search resulted in 237 studies. The first stage involved screening using Scopus filters (year, document type, publication stage, and language) to remove records clearly outside the defined criteria. Unavailable papers and duplicates were also removed. The total screening reduced the dataset to 204 studies.

A CSV export from Scopus was then imported into Google Sheets for the selection stage. The columns were reordered to prioritize the most relevant information, while less essential ones were removed. Filtration drop-down options were added as new columns to track statistics and ease the sort/filtration later. Additional columns were added to support structured assessment: *Programming* (yes/no), *Wordcount* (<5000/OK), *Empirical Data* (yes/no), and total evaluation (relevant/maybe/irrelevant/not available). A column for comments was also added to keep track of decisions. All the columns used for the selection stage are presented in Table 2.2.2.

| Column | Value Description |
|---|---|
| Title | Paper title |
| Abstract | Abstract used for screening |
| Evaluation | relevant / maybe / irrelevant |
| Programming | Yes/No flag for programming-related content |
| Word count | <5000/OK for under/over 5000 words (excl. abstract/refs) |
| Empirical data | Yes/No flag for empirical evidence |
| Comments | Notes on screening decisions |
| Year | Publication year |
| Authors | Author(s) list |
| Cited by | Citation count from export |
| DOI | Digital Object Identifier |
| Link | Stable URL to record/full text |
| Document type | Article / Conference paper |

**Table 2.2.2:** Selection Criteria

The spreadsheet formed the basis for the selection stage. To establish a common understanding of how each criterion should be applied, the first ten studies were evaluated collaboratively. For each of these papers, all columns in the spreadsheet were completed together, ensuring agreement on how to classify word count, empirical data, programming content, and overall relevance.

After this calibration phase, the remaining studies were evaluated individually. For each paper, all columns were filled out, starting with the objective criteria (word count, empirical data, programming content), followed by the relevance evaluation based on title and abstract. After this individual process, the papers marked as maybe were revisited in a second collaborative round, since these were the cases where uncertainty remained during the first pass. The total selection process reduced the number of studies from 204 to 51.

These steps correspond directly to the staged reductions illustrated in the PRISMA flow diagram (Figure 2.2.2), which summarizes the screening and selection phases and shows how the initial 237 records were reduced to 51 eligible studies.

### 2.2.5 Data Extraction and Coding

The data extraction and coding of the literature turn each study into a consistent record ready for analysis. Making it easy to compare eye-tracking metrics, participants, and feedback designs across papers. This section presents our coding process, which enforces standardization, making this study not only reliable but also reproducible.

After a closer review of the included papers, three additional studies were excluded because they did not fit the programming-related scope of the review. Although these papers passed the initial selection stages, they were removed during data extraction when it became clear that they lacked sufficient relevance to programming or debugging tasks. This final adjustment ensured full alignment between the inquiry question and the dataset, leaving a total of 48 papers after coding was completed.

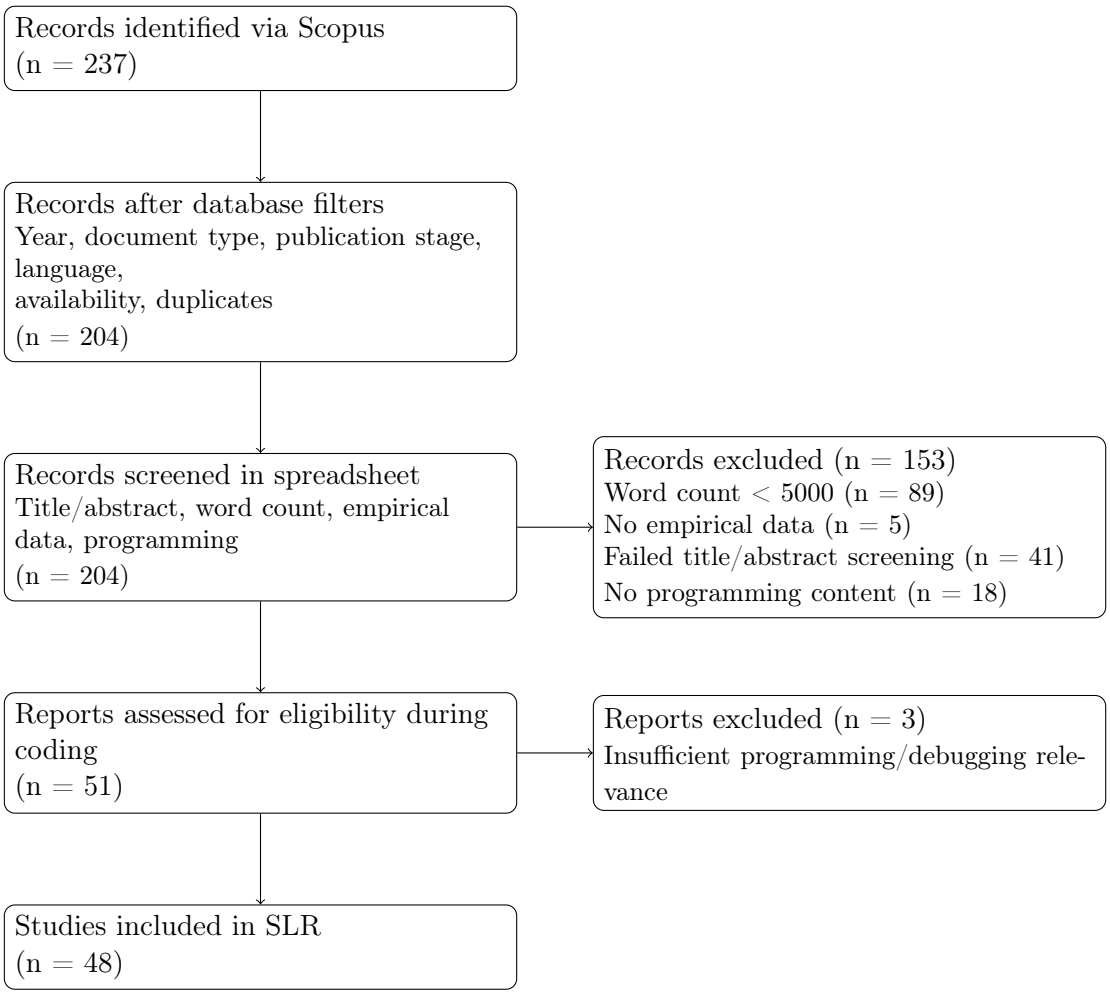



**Figure 2.2.2:** PRISMA-style flow of the study identification, screening, and selection process.

#### 2.2.5.1 Coding Framework

The coding framework defines the variables extracted from each study and ensures that data were captured consistently across the included literature. In line with the recommendations of [16], the extraction schema was designed in advance to reduce bias and support replicability. The structure and categories of the variables were selected to cover all information required to address the review's inquiry question.

The variables were developed during a collaborative workshop with the supervisor and co-supervisor, both of whom have experience with SLR methodology and eye-tracking research in programming education. Their input ensured that the coding structure aligned with the objectives of the review and reflected established knowledge in the field. In addition, each variable, except for the metadata fields, was defined to connect to at least one SIQ to ensure relevance to the inquiry question for the SLR.

The final coding framework was reviewed collaboratively to ensure a shared understanding of each variable and to promote consistency across reviewers during extraction. This refinement aligns with the guidance of [16], which recommends that extraction schemas be validated by multiple researchers or under supervision to ensure transparency and reliability.

#### 2.2.5.2 Codebook and Variable Definitions

Table 2.2.3 presents the complete set of variables used in the review. Each field corresponds to a specific SIQ, with definitions or possible values. This structured codebook supports transparency and enables other researchers to reproduce the data extraction process or extend the dataset in future reviews.

#### 2.2.5.3 Extraction Procedure

The extraction process followed a semi-automated approach. Metadata were imported automatically from the CSV file exported from Scopus, including title, authors, year, and DOI. The remaining variables were extracted manually by the reviewers. The first five papers were coded collaboratively to establish a shared understanding of each variable and to calibrate coding decisions. The remaining studies were then coded individually.

During the manual extraction process, ChatGPT (versions 4 and 5) was used as an auxiliary tool to highlight potentially relevant text segments aligned with the variables in the coding scheme. Its outputs were treated as suggestions and were systematically reviewed and verified during the final data extraction. This approach enhanced both efficiency and consistency of the coding process while maintaining full human oversight and methodological transparency.

#### 2.2.5.4 Inter-Coder Reliability

Although the review did not employ formal inter-rater reliability metrics, reliability was addressed through collaborative calibration and continuous discussion during the extraction phase. Unclear cases were commented on and revisited to ensure agreement. This process partially aligns with the practice recommendations of [16], which emphasize consensus-building and documentation as valid methods for maintaining data consistency in reviews. For later reviews, a method to assess the inter-researcher consistency should have been applied more thoroughly [16].

#### 2.2.5.5 Data Cleaning and Consolidation

Having a consistent and cleanly coded dataset with documents was necessary to compare the different studies. After coding, the dataset was carefully reviewed to identify missing values, inconsistent naming, and formatting errors. Firstly, if a variable was not stated or mentioned in the study, it was defined as “not stated” or “N/A”, depending on the column. For inconsistent naming, a second iteration was done to ensure consistent values and formats. This ensured that comparisons between studies were possible.

#### 2.2.5.6 Standardization of Heterogeneous Variables

The included studies reported a diverse range of performance measures, gaze metrics, and analytical approaches. To manage this variability, a secondary level of coding was applied to group these heterogeneous variables into broader analytical dimensions. This standardization improved comparability across studies and allowed the data to be analysed in a more structured way.

| **Field** | **Description / Possible Values** | **SIQ(s)** |
|---|---|---|
| Title | Title of the Paper | - |
| Year | Year study was published | - |
| Task description | (e.g., comprehension, find the bug, debugging, find the error, find the flaw, programming, code review) | SIQ1 |
| Programming Language | If presented, the programming language used in the stimuli. | SIQ1 |
| Population | (College students, Elementary school, Faculty members, Graduate, K-12, Non-programmers, Novice programmers, PhD, Professional Developers, Undergraduate, University Students) | SIQ2 |
| Sample size | The number of valid subjects in the study | SIQ2 |
| Number of people tracked per session | (1, 2, 3) | SIQ1 |
| Unit of analysis | (individual, pair, team) | SIQ1 |
| Type of eye-tracking equipment | E.g. Tobii xx | SIQ3 |
| Eye-tracking metrics used | (FIX, SAC, AOI, SEQ, PUP, VIS, OTH) | SIQ3 |
| Performance measurement | (EFFECT, TIME, EFFORT, SUBJ, OTHER) | SIQ4 |
| Method of analysis | (DES, INF, REG, CORR, ML, SEQ, QUAL, VIS, OTH) | SIQ5 |
| Feedback provided | (yes, no) | SIQ6 |
| Type of feedback | (if yes) what feedback was provided? | SIQ6 |
| Feedback delivery | (proactive, reactive) | SIQ6 |
| Feedback trigger | (which measurement and what levels) | SIQ6 |
| Research question(s) | Copy-paste from article | SIQ7 |
| Research objective | Copy-paste if explicitly stated | SIQ7 |
| Limitations | Copy-paste if mentioned | SIQ8 |
| Future work | Copy-paste if mentioned | SIQ8 |
| URL | Scopus URL | - |
| DOI | DOI value | - |

**Table 2.2.3:** Template for data extraction and coding used in the SLR on eye-tracking for debugging with novice programmers. Each field corresponds to a coding dimension in the review, and an inquiry question

To handle the variation in performance-related measures, all reported metrics were consolidated into conceptual categories that reflect their underlying purpose rather than the specific labels used in each study. This approach aligns with established practices in empirical software engineering for data reduction and coding [17]. The resulting performance codes were informed by both the extracted data and established knowledge in the field, and are presented in Table 2.2.4.

| **Code** | **Dimension** | **Meaning / Focus** |
|---|---|---|
| EFFECT | Effectiveness / Correctness | How accurate or successful the participant was in solving the task |
| TIME | Efficiency / Time | How quickly the participant performed or completed the task |
| EFFORT | Cognitive / Visual Effort | How much mental or visual effort the participant exerted |
| SUBJ | Subjective Evaluation | Self-reported perceptions or attitudes |
| OTHER | Other or Mixed | Composite scores or unclear metrics |

**Table 2.2.4:** Coding scheme for evaluation of performance metrics.

Similarly, to handle the diversity of gaze-based measures, all eye-tracking metrics reported were coded into higher-level analytical categories that reflect common analytical foci across studies, as shown in Table 2.2.5.

| **Code** | **Dimension** | **Meaning / Focus** |
|---|---|---|
| FIX | Fixation-based metrics | Describe gaze stability and visual attention on specific areas or objects |
| SAC | Saccade / Transition metrics | Describe gaze shifts and visual scanning behaviour |
| AOI | Area-of-Interest (AOI) metrics | Describe distribution of attention across predefined regions (lines, methods, variables, etc.) |
| SEQ | Sequence / Scanpath metrics | Describe temporal order and sequential patterns of fixations |
| PUP | Pupil / Physiological metrics | Reflect cognitive load or arousal |
| VIS | Visualization / Aggregate outputs | Visual or summary representations used for interpretation |
| OTH | Other or Compound | Mixed or unclear measures, combined indices |

**Table 2.2.5:** Categories of gaze and physiological metrics used in the reviewed studies.

Finally, all reported analytical methods were coded into standardized methodological dimensions to facilitate comparison of quantitative and qualitative ap-

proaches across studies. The coding scheme is presented in Table 2.2.6.

| Code | Dimension | Meaning / Focus |
|---|---|---|
| DES | Descriptive / Exploratory Analysis | Reporting distributions and summary statistics; visual inspection |
| INF | Inferential Statistics / Hypothesis Testing | Comparing groups or conditions statistically |
| REG | Regression / Model-Based Analysis | Modeling effects or relationships between variables |
| CORR | Correlation / Association Analysis | Measuring strength of relationships |
| ML | Machine-Learning / Predictive Modeling | Using algorithms for classification or prediction |
| SEQ | Sequence / Transition Analysis | Temporal or order-based analyses of gaze or actions |
| QUAL | Qualitative / Mixed Methods | Interpretive or text-based analysis |
| VIS | Visualization / Graphical Comparisons | Graph-based or visual analytical tools |
| OTH | Other / Compound | Combined or unspecified approaches |

**Table 2.2.6:** Categories of analytical and statistical methods used in the reviewed studies.

#### 2.2.5.7 Example of Coded Study

Table 2.2.7 illustrates a simplified example of a coded study, showing how raw publication information was transformed into structured data in the review dataset.

### 2.2.6 Thematic Synthesis

In consultation with the supervisors, the included studies were organized into five analytical themes derived from recurrent concepts, methodological patterns, and reported outcomes. In line with the guidance of [16], data synthesis in a systematic literature review should go beyond mere description by integrating findings to identify overarching trends, similarities, and contrasts across studies. The purpose of the thematic synthesis is therefore to integrate results across heterogeneous designs, tasks, and operationalizations, revealing cross-study regularities and meaningful differences. Rather than cataloging the papers, this synthesis aggregates conceptually related findings into clusters.

With the screening, extraction, and coding processes completed, the final dataset consisted of 48 studies that met all inclusion criteria. While the methodology outlines how the corpus was constructed, it does not yet characterize what the studies actually contain. The next section therefore presents the descriptive findings, providing an overview of the key features of the included studies, such

| **Variable** | **Example Value** |
|---|---|
| Task description | Debugging |
| Programming Language | Java |
| Population | Undergraduate |
| Sample size | 24 |
| Number of people tracked per session | 1 |
| Unit of Analysis | Individual |
| Eye-tracking equipment | Tobii Pro X3-120 |
| Eye-tracking measurement | fix,sac |
| Performance measurement | effect, time |
| Method of Analysis | vis, inf |
| Feedback provided | No |
| Type of feedback | N/A |
| Feedback delivery | N/A |
| Feedback trigger | N/A |
| Research questions | How do cognitive and emotional states vary with performance in programming tasks? |
| Research objective | Examine cognitive and emotional differences between novices and beginners using multimodal psychophysiological data during programming. |
| Limitations | Laboratory setting limits ecological validity |
| Future work | Extend to authentic classroom settings |

**Table 2.2.7:** Example of a coded study record.

as task types, populations, metrics, and analytical approaches—before moving on to deeper synthesis.

## 2.3 Descriptive Findings

This section summarizes the descriptive characteristics of the 48 studies included in the final corpus. The aim is to provide an overview of the empirical landscape surrounding eye-tracking research in programming and debugging contexts before later moving on to the thematic synthesis. The descriptive analysis focuses on four dimensions central to understanding methodological and contextual diversity in the field: (i) study characteristics such as publication year, task type, and programming language; (ii) population characteristics, including sample size, educational background, and number of participants tracked per session; (iii) analytical and measurement approaches, and (iv) the experimental setups used across studies.

Together, these descriptive findings establish the foundation for interpreting patterns, methodological tendencies, and gaps in the subsequent synthesis. A complete overview of all extracted and coded values for every included study is provided in Appendix A.1.

### 2.3.1 Study Characteristics

This subsection addresses SIQ1 by summarizing how the included studies operationalize debugging and related tasks, including their task types, programming languages, and experimental setups. These characteristics provide the foundation for understanding how visual attention and cognitive processes are examined across the literature.

#### 2.3.1.1 Publication Year Distribution

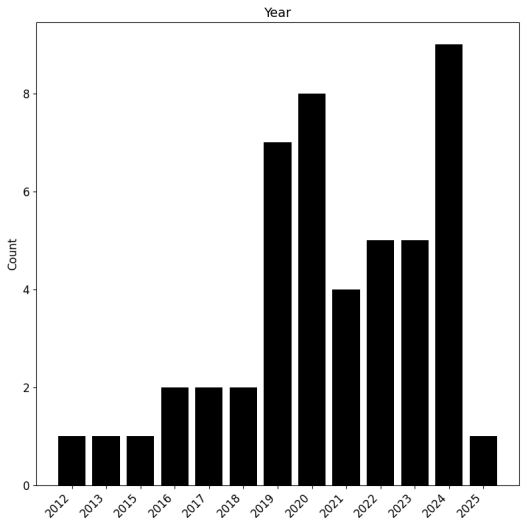
A total of 48 studies published between 2012 and 2025 were included. The output is strongly skewed toward recent years, with most studies published between 2019 and 2025 ($n$ = 39, 81.3%). The number of publications peaks in 2024 ($n$ = 9, 18.8%), with secondary highs in 2020 ($n$ = 8, 16.7%) and 2019 ($n$ = 7, 14.6%). The mid-period shows moderate activity (2021: $n$ = 4; 2022–2023: $n$ = 5 each), while earlier years are sparse (2012, 2013, and 2015: $n$ = 1 each; 2016–2018: $n$ = 2 each). Overall, the trend indicates a growing research interest in the topic since 2019, with one study identified in 2025 [18]. The yearly distribution is shown in Figure 2.3.1(a).

#### 2.3.1.2 Task Description

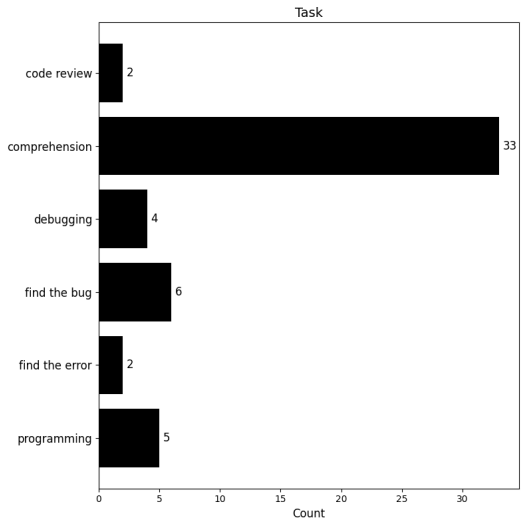
Tasks were coded at the task instance level, as several studies included multiple experimental tasks. In total, 52 task instances were recorded. The majority involved code comprehension ($n$ = 33, 63.5%), followed by bug finding ($n$ = 6, 11.5%), programming ($n$ = 5, 9.6%), and debugging ($n$ = 4, 7.7%; [19, 5, 20, 18]). Less frequent were code review ($n$ = 2, 3.8%; [21, 22]) and error identification tasks ($n$ = 2, 3.8%; [23, 24]). Figure 2.3.1(b) visualizes this distribution.

#### 2.3.1.3 Programming Language

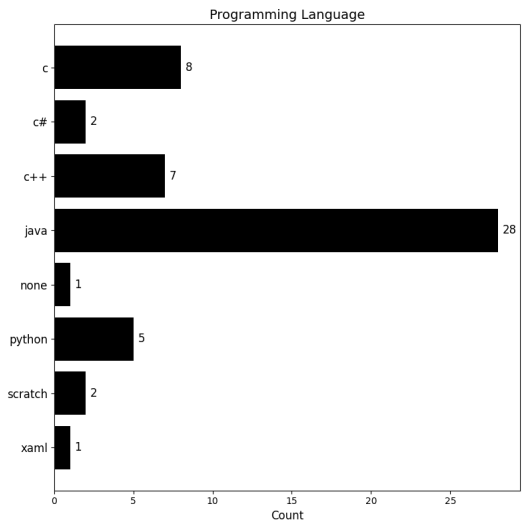
Across the included studies, 54 programming-language instances were recorded, as some studies employed more than one language. Java dominated the corpus ($n = 28$, 51.9%), followed by C ($n = 8$, 14.8%), C++ ($n = 7$, 13%), and Python ($n = 5$, 9.3%). Only a few studies used C# ($n = 2$, 3.7%; [25, 26]) or Scratch ($n = 2$, 3.7%; [27, 28]) while XAML [26] and language-independent ("None") tasks [11] were each represented by a single study. This dominance of Java and C-family languages suggests that most findings generalize to statically typed, C-like syntaxes, while dynamic and block-based environments remain underrepresented.




(a) Publication Year Distribution




(b) Task Description




(c) Programming Language

**Figure 2.3.1:** Overview of study characteristics across the 48 included studies

Overall, the observed variation in task types, the dominance of Java and C-family languages, and the prevalence of controlled laboratory setups collectively explain how debugging-related tasks are operationalized across studies, thereby addressing SIQ1.

### 2.3.2 Population Characteristics

This subsection addresses SIQ2 by examining the populations involved in these studies and how sample characteristics may influence insights about novice debugging behaviour. Mapping participant backgrounds, sample sizes, and group configurations clarifies the empirical basis from which conclusions about novices are drawn.

#### 2.3.2.1 Sample Size

As shown in Figure 2.3.2(a), sample sizes across the 48 studies ranged from 7 to 207 participants ($Mean = 49$, $Median = 34$). The distribution is strongly right-skewed, with more than half of the studies including between 20 and 63 participants. Small samples of 10–50 participants formed the majority of the corpus ($n = 28$, 58.3%), followed by medium-sized studies with 51–100 participants ($n = 9$, 18.7%). Only a few studies exceeded 100 participants ($n = 5$, 10.4%), with three reporting very large samples above 150 [29, 18, 30]. This pattern reflects the practical constraints of eye-tracking research, where instrumentation, calibration, and data-collection demands make large cohorts difficult to manage.

#### 2.3.2.2 Participants Tracked per Session

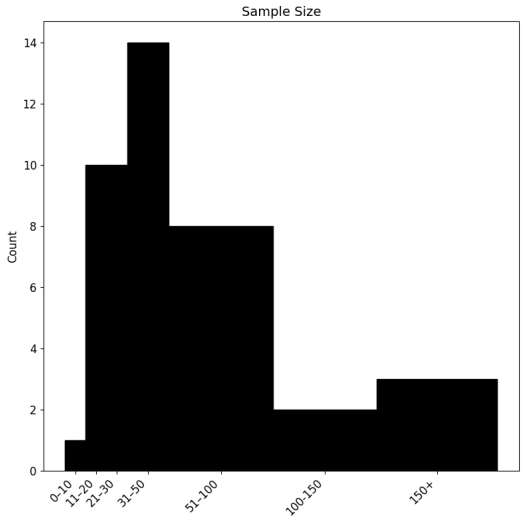
As shown in Figure 2.3.2(b), most studies tracked participants individually. Single-participant setups dominated the corpus ($n = 43$, 89.6%), while a smaller subset tracked two or more participants simultaneously ($n = 6$, 12.5%). One study did not report the number of participants tracked [31]. This strong preference for single-user setups aligns with the methodological focus on isolating individual cognitive and attentional processes.

#### 2.3.2.3 Population Educational Background

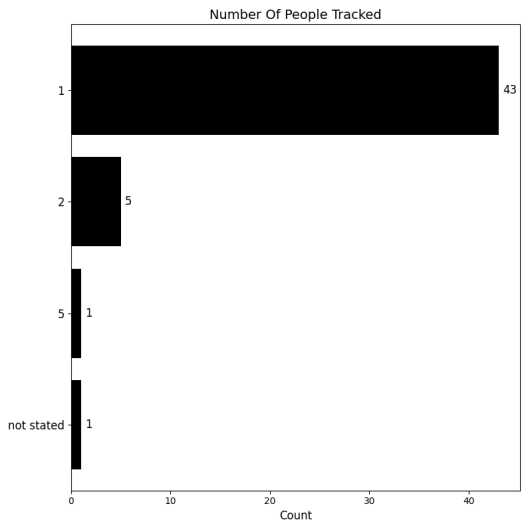
As shown in Figure 2.3.2(c), participants were primarily drawn from higher education settings. Undergraduate students constituted the largest group ($n = 33$, 68.8%), followed by graduate students ($n = 21$, 43.8%) and professional developers ($n = 14$, 29.2%). Smaller populations included novice programmers ($n = 5$, 10.4%) and PhD students ($n = 4$; 8.3%; [32, 33, 34, 35]). Other infrequent categories were college students [36, 37, 21], faculty members [38, 30], K–12 learners [27], and non-programmers [30], (each $n \leq 3$). Overall, this distribution suggests a strong academic orientation, with most studies focusing on learners in computer science education.

#### 2.3.2.4 Unit of Analysis

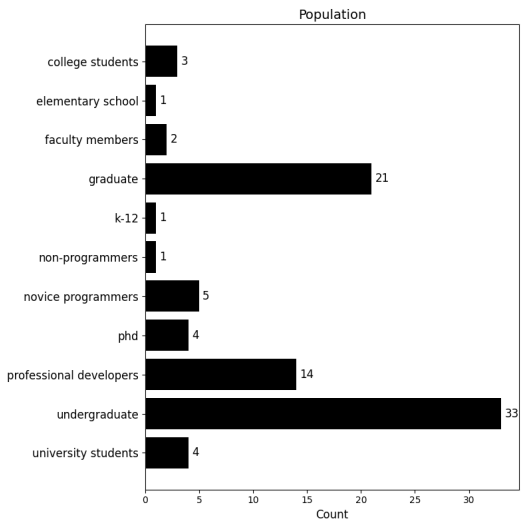
As shown in Figure 2.3.2(d), nearly all studies analyzed data at the individual level. Individual-based analysis dominated the corpus ($n = 46$, 95.8%), whereas only a small number examined pairs ($n = 4$, 8.3%; [18, 21, 20, 37]). This pattern aligns with the prevalence of single-participant tracking sessions and indicates a focus on personal cognitive and visual behaviour rather than collaborative interactions.



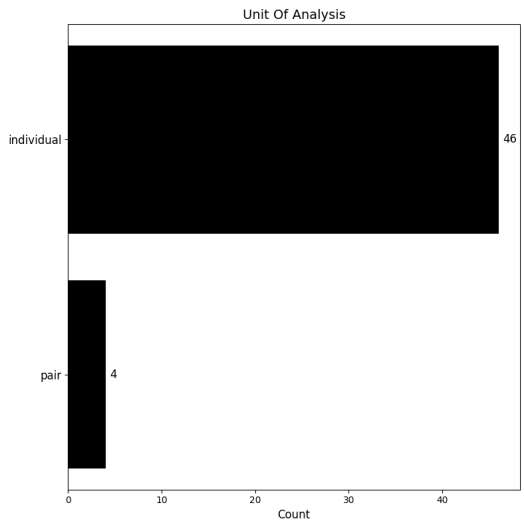

(a) Sample Size




(b) Number of People Tracked per Session




(c) Educational Background




(d) Unit of Analysis

**Figure 2.3.2:** Overview of population characteristics across the 48 included studies.

In sum, the variation in sample sizes, the dominance of undergraduate and graduate student populations, and the strong focus on individual participants collectively describe the populations these studies rely on and clarify how such sample characteristics shape the insights that can be drawn about novice debugging behaviour, thereby addressing SIQ2.

### 2.3.3 Analytical and Measurement Approaches

This subsection addresses SIQ3, SIQ4, and SIQ5 by outlining how studies capture visual attention and cognitive effort, how they measure debugging performance, and which analytical methods are used to interpret these data. Together, these

measurement and analysis choices reveal how attention, cognitive load, and performance are connected in existing research.

#### 2.3.3.1 Metrics

As shown in Figure 2.3.3(a), fixation-based measures dominated the corpus ($n$ = 40, 83.3%). These typically included fixation count, duration, and distribution across areas of interest (AOIs), forming core indicators of attentional focus and cognitive processing. AOI-related metrics were also widely used ($n$ = 16, 33.3%), reflecting how researchers linked gaze data to specific code structures such as lines, variables, or methods.

A smaller subset of studies employed saccade-based measures ($n$ = 14, 29.2%), often to capture scanning patterns or visual search efficiency. Sequence-based metrics appeared with similar frequency ($n$ = 13, 27.1%), typically to analyze temporal or logical order in gaze transitions during reading or debugging. Pupil-based measures were less common ($n$ = 8, 16.7%), used primarily to infer cognitive load from changes in pupil diameter. Only a few studies used visualization-oriented metrics ($n$ = 3, 6.3%; [33, 39, 40]), where heatmaps or gaze plots supported qualitative interpretation rather than quantitative analysis.

Overall, fixation and AOI metrics were the most prevalent, indicating a clear methodological emphasis on localized and count-based indicators of visual attention, while temporal, physiological, and visualization-focused measures were used far less frequently.

#### 2.3.3.2 Performance Evaluation

How the included studies evaluated participants' performance outcomes was analyzed using five main categories (Table 2.2.4. As shown in Figure 2.3.1(d), most evaluations focused on effectiveness or correctness ($n$ = 38, 79.2%), emphasizing accuracy-based outcomes such as correct solutions, bug detection success, or comprehension scores. Efficiency measures ($n$ = 18, 37.5%) were the second most frequent, typically operationalized as task completion time or response latency.

By contrast, subjective evaluations ($n$ = 8, 8.3%) and effort-related metrics ($n$ = 3, 6.3%; [19, 38, 30]) appeared much less frequently, typically capturing perceived task difficulty, cognitive load, or visual effort. Finally, a small subset of studies ($n$ = 7, 14.6%) reported mixed, missing, or composite indicators that could not be clearly classified under a single category. Among these were four cases labeled as "not reported" [34, 41, 35, 42], one case labeled as "not used" [43], and two cases of "combined measurement" [44, 28].

Overall, the corpus reveals a methodological preference for objective and outcome-oriented metrics (effectiveness and time), whereas self-reported or process-oriented measures (subjective and effort) remain relatively scarce. This imbalance suggests that most studies prioritize quantifiable performance outcomes over subjective experiences or cognitive processes of participants.

#### 2.3.3.3 Methods of Analysis

The distribution of analytical methods, as shown in Figure 2.3.2(b), highlights a strong preference for inferential statistics ($n$ = 38, 79.2%), including tests such as

ANOVA, $t$-tests, and non-parametric equivalents for comparing gaze metrics across experimental conditions or expertise levels. Visual analysis ($n = 11$, 23%) and sequence analysis ($n = 10$, 20.8%) followed, often used for qualitative inspection of gaze paths, heatmaps, or temporal gaze transitions.

Correlation ($n = 9$, 18.8%) and regression ($n = 5$, 10.4%) analyses were applied to identify relationships between gaze variables and performance outcomes, while qualitative interpretations ($n = 8$, 16.7%) incorporated video inspection or thematic coding to contextualize gaze behaviour.

More advanced techniques, such as machine-learning approaches ($n = 4$, 8.3%; [45, 41, 46, 37]), were relatively rare but more frequent in the later years, and typically used for classifying expertise or predicting performance from gaze data.

One study did not fit neatly into the predefined categories and was therefore classified as "other." It employed G*Power and Statistica 12 for supplementary analyses in addition to standard inferential statistics [25].

This distribution underscores a methodological imbalance: most studies rely on conventional inferential testing rather than integrative or data-driven models, suggesting room for more diverse and computationally sophisticated approaches in future research.

#### 2.3.3.4 Equipment

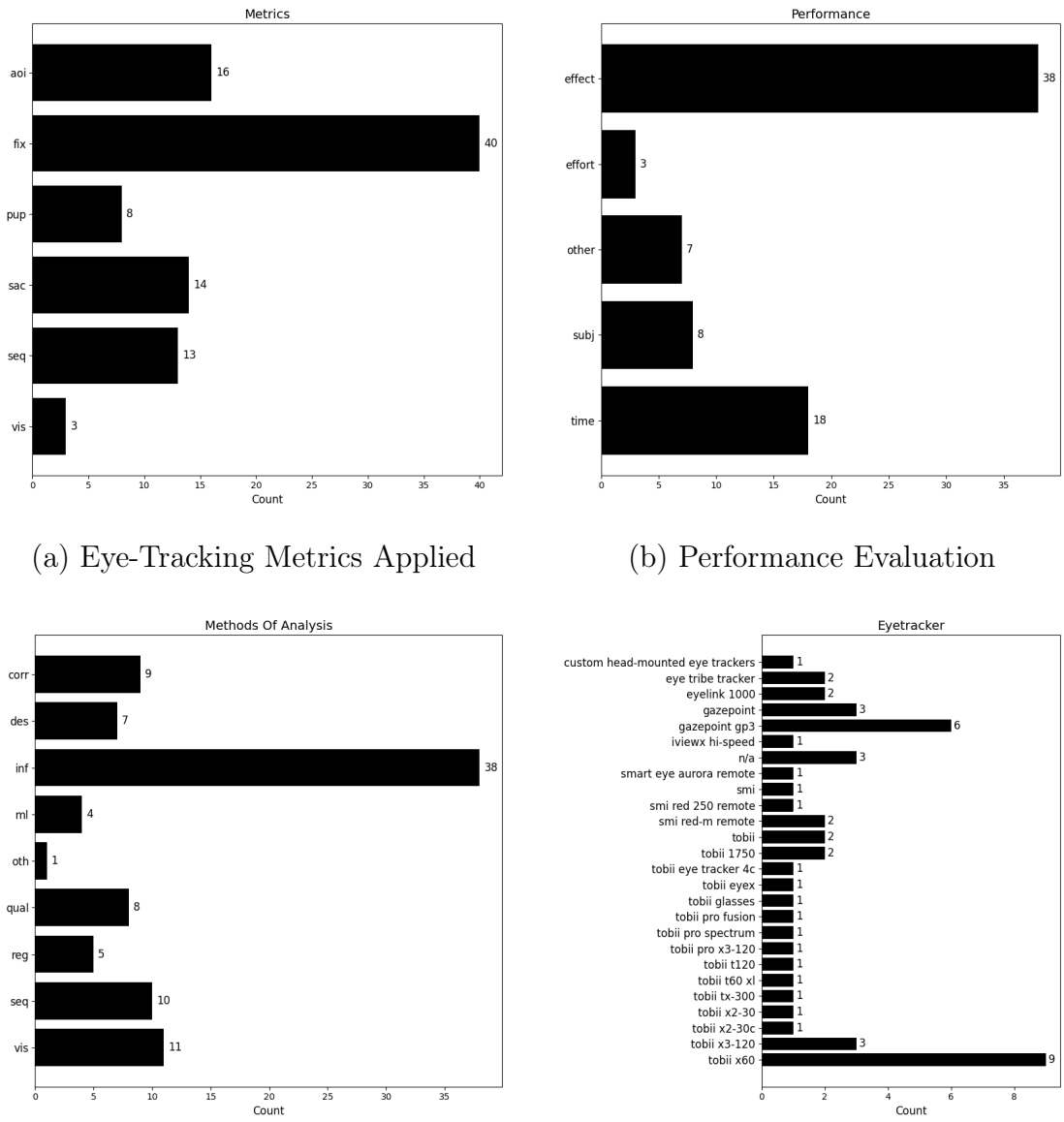
Eye-tracking hardware varied considerably across studies (Figure 2.3.2(c)), yet Tobii devices were by far the most common, appearing in more than half of all cases. The Tobii X60 ($n = 9$, 18.8%) was the single most frequently used model, followed by the Gazepoint GP3 ($n = 6$, 12.5%). Other Tobii models—including the X3-120, T1750, TX-300, and Pro Spectrum—appeared occasionally.

A smaller number of studies employed SMI ($n = 4$, 8.3%; [47, 41, 27, 29]) or EyeLink ($n = 2$, 4.2%; [5, 23]) trackers, or custom head-mounted systems ($n = 1$, 2%; [21]). Three studies did not specify the device used [48, 31, 33]. The prevalence of Tobii hardware reflects its accessibility and ease of integration with programming-related setups, while the underrepresentation of high-speed or head-mounted systems suggests that most experiments prioritized ecological validity and usability over sampling precision.

Overall, the dominance of fixation and AOI metrics, the strong reliance on objective correctness and time-based performance evaluations, and the widespread use of conventional inferential statistics together with the prevalence of Tobii devices collectively show how studies capture visual attention and cognitive effort (SIQ3), how they assess debugging performance (SIQ4), and which analytical approaches they use to interpret these data (SIQ5).

### 2.3.4 Summary

Across the descriptive analysis, the findings clarify how the included studies answer the guiding sub-questions. SIQ1 is addressed by showing how debugging-related tasks are operationalized through recurring patterns in task type, programming language, and experimental setup. SIQ2 is answered by mapping the populations involved, highlighting the dominance of student participants, modest sample sizes, and a strong focus on individual-level analysis. SIQ3, SIQ4, and SIQ5 are




(a) Eye-Tracking Metrics Applied

(b) Performance Evaluation

(c) Methods of Analysis

(d) Eye-Tracking Equipment

**Figure 2.3.3:** Overview of analytical and measurement approaches across the 48 included studies

addressed by identifying the most common metrics for capturing visual attention and cognitive effort, the outcome-oriented measures used to evaluate debugging performance, and the prevailing analytical methods used to interpret these data.

Together, these findings outline the methodological landscape that underpins current eye-tracking research in programming and debugging. They reveal where the field is concentrated, where it is limited, and where methodological variation may influence the conclusions drawn about novice cognition and performance. Building on this foundation, the following section moves beyond description and synthesizes these patterns to identify conceptual trends, methodological tendencies, and recurring limitations that define the current state of the art.

## 2.4 Thematic Synthesis

This section integrates evidence across the 48 studies by organizing results into (i) conceptual themes, (ii) methodological challenges, and (iii) opportunities for future work. It first synthesizes the state of the art, then summarizes cross-cutting limitations, and finally outlines actionable research directions.

### 2.4.1 State of the Art

This subsection also addresses SIQ7 by examining how the studies' stated research questions and objectives align with their operationalizations of debugging, attention, and measurement. It synthesizes the main empirical evidence across five recurring themes that define the current state of the art in eye-tracking research on programming: problem-solving strategies, code comprehension, expertise differences, cognitive load and performance, and collaboration. These themes illustrate how visual attention data have been used to explain individual reasoning, comprehension, and joint coordination in programming tasks. Most studies employ short Java or C-like code snippets, comprehension or debugging tasks, and fixation-based analyses, which collectively shape the comparability and analytical focus of the existing literature. An overview of all themes and their corresponding papers is provided in Table 2.4.1.

#### 2.4.1.1 Problem-Solving and Visual Programming Strategies

This theme covers studies that investigate how visual representations, layouts, and visualization tools influence novice programmers' understanding and problem-solving ability. These works emphasize pedagogical interfaces, visual cues, and representational design as mechanisms for supporting conceptual understanding and reducing cognitive load. The overarching goal is to explore how visual stimuli and design decisions can scaffold cognitive processing and enhance learning outcomes among beginners.

Talsma et al. [49] examined how block highlighting affects the reading behaviour of novice programmers performing code-comprehension tasks. Although comprehension scores did not significantly differ across highlighting conditions, eye-tracking data revealed clear visual-structuring effects: participants subconsciously followed block boundaries, suggesting that visually chunked code supports more organized reasoning.

Complementing this, Obaidellah et al. [11] used Area-of-Interest sequence analysis to classify eye-movement strategies during algorithmic problem solving. They identified distinct patterns distinguishing efficient from inefficient solvers, linking differences in gaze transitions to underlying mental schemas and problem-solving efficiency.

Finally, Bednarik [50] explored how programmers of varying expertise coordinate multiple visual representations during debugging, showing that novices frequently alternated between code and graphical views, whereas experienced participants maintained more stable and goal-directed attention across representations.

Collectively, these studies demonstrate that visual and representational design substantially shape how novices allocate attention, form mental models, and

| Theme | Summary | Included Papers |
|---|---|---|
| **Problem-Solving and Visual Programming Strategies** | Focuses on how visual representations guide novices' attention and reasoning. Eye-tracking results show that structured visual cues help beginners process information more effectively. | [49] [11] [27] [50] [45] [40] |
| **Code Comprehension and Eye Tracking** | Investigates how programmers read and understand code through fixation patterns and visual attention. Code characteristics and task demands strongly shape cognitive effort. | [51] [26] [46] [52] [53] [48] [42] [39] [23] [25] [30] [32] |
| **Expertise and Visual Attention Differences** | Examines how experts and novices differ in visual attention during programming. Experts show more efficient and goal-directed gaze patterns, while novices exhibit higher cognitive effort and less structured viewing. | [47] [29] [54] |
| **Cognitive Load and Developer Performance** | Focuses on how cognitive load influences programming accuracy, efficiency, and debugging success. Clearer code structure reduces mental effort and supports faster, more confident problem-solving. | [55] [18] [56] [38] [28] [57] [58] |
| **Collaboration and Gaze Patterns in Pair Programming** | Examines how pairs coordinate attention during collaborative programming. Productive pairs maintain flexible, well-timed gaze coordination, while unsynchronized patterns indicate weaker collaboration. | [20] [36] [37] [59] [43] |

**Table 2.4.1:** Overview of themes and corresponding papers included in the thematic synthesis.

regulate cognitive effort during programming tasks. Eye-tracking evidence consistently indicates that structured and pedagogically informed visualizations, such as highlighting, color cues, and schematic layouts, support fragmentation and comprehension, whereas poorly aligned or overly assistive interfaces may encourage shallow engagement. This body of work positions visualization as a powerful pedagogical and analytical lens for understanding and improving problem-solving in early programming education.

#### 2.4.1.2 Code Comprehension and Eye tracking

This is the largest theme, covered by 12 different papers, focusing on how programmers comprehend and interpret code using eye-tracking techniques. The studies analyze fixation patterns, reading sequences, and visual attention to understand how programmers process code during comprehension tasks. Many compare different comprehension contexts (e.g., reading vs. summarizing) or programming languages (e.g., Python vs. Java). The goal is to map eye movement data to the cognitive processes involved in reading and understanding the source code.

Binkley et al. [30] found that the identifier naming style influences both visual effort and comprehension. Their eye-tracking data showed that programmers made more and longer fixations on camelCase identifiers than on underscore, indicating greater visual effort for the former. However, across comprehension tasks, camelCase yielded slightly higher accuracy, leading the authors to conclude that it is generally more effective, particularly for novice programmers.

Furthermore, Segedinac et al. [46] assessed Python code readability using eye tracking data from 90 students. They found that longer gaze times on loops, branching, and function calls correlated with lower readability ratings, indicating that increased visual effort signals reduced readability. A machine-learning model trained solely on these gaze-based features achieved an F-measure of 0.85, demonstrating that eye tracking data can effectively predict perceived code readability.

Finally, Karas et al. [51] compared reading and writing of Java code summaries, finding that programmers' visual attention depends on the comprehension task. When writing summaries, participants spent longer and focused more on parameters, variable declarations, and method calls, reflecting deeper semantic processing, whereas reading pre-written summaries elicited shorter, surface-level fixations on declarations.

Together, these studies illustrate how eye-tracking provides fine-grained insights into programmers' cognitive processes during code comprehension. Across tasks ranging from identifier reading and code summarization to assessing readability, fixation-based measures consistently reflected underlying cognitive effort and information processing. The results show that visual attention is shaped by both code characteristics (e.g., naming style, control structures) and task demands (e.g., reading vs. writing), suggesting that comprehension involves adaptive strategies that balance visual effort with semantic understanding. Collectively, these findings demonstrate the value of eye-tracking for linking observable gaze behaviour to deeper cognitive mechanisms in programming.

#### 2.4.1.3 Expertise and Visual Attention Differences

This theme explores differences between expert and novice programmers, primarily focusing on eye movement metrics such as fixation counts, gaze duration, and saccade patterns. The research aims to identify behavioural signatures of expertise, demonstrating how experts focus their visual attention more efficiently and purposefully. Some studies even develop models or measures to quantify expertise based on gaze data or to predict proficiency levels.

Busjahn and Paterson [47] compared experts and novices reading Java programs and found that experts identified the program's entry point earlier and

followed a more structured reading sequence, reflecting goal-directed comprehension and efficient navigation of code structure.

Using pupillometry, Aljehane et al. [29] observed that novices showed greater pupil dilation and longer fixation durations, indicating higher cognitive workload and less efficient information processing during comprehension tasks.

In contrast, Jessup et al. [54] found that experts sometimes made more, but shorter, fixations when reading larger codebases, suggesting a flexible visual strategy where experienced programmers scan more widely while maintaining concise and targeted focus within relevant code regions.

Together, these studies demonstrate that expertise manifests through structured, adaptive, and purposeful visual attention, paired with reduced cognitive effort even in complex programming tasks. At the same time, differences in fixation-based outcomes across studies indicate that expertise effects are context dependent, shaped by factors such as code length, scrolling behaviour, and task design. This underscores the importance of employing consistent stimuli and complementary physiological measures (e.g., pupil dilation) when evaluating expertise through eye-tracking.

#### 2.4.1.4 Cognitive Load and Developer Performance

These papers address cognitive load and performance factors in programming. They examine how code structure, complexity, and readability rules (like minimizing nesting) affect understanding, error detection, and developer confidence. Many adopt experimental approaches combining behavioural and eye-tracking data to estimate mental effort and identify ways to optimize code presentation or tool design for better comprehension and reduced cognitive strain.

Park et al. [55] replicate readability rules with eye tracking and link gaze to performance. Minimizing nesting is associated with lower visual effort (fewer/shorter fixations when normalized), less time, and higher confidence, while accuracy differences are small and often non-significant. Logically incorrect code induces more regressions, indicating difficulty. Avoiding do-while has weaker effects on time/visual navigation but yields higher accuracy. Overall, readability choices modulate cognitive load and, in turn, time and correctness.

Furthermore, Mangaroská et al. [56] use multimodal data during code debugging and show that attention, convergent thinking, and frustration correlate positively with performance, while cognitive load, memory load, and boredom correlate negatively. These relations persist across phases from reading to change–test–fix, with confusion sometimes aligning with productive progress and delight not predicting better outcomes. The results emphasize that elevated load tracks slower or less successful debugging, whereas focused effort signatures accompany better performance.

Finally, Sharma and Giannakos [18] validate the carry-forward effect across game-based learning and both individual and collaborative debugging. Measures such as cognitive load and joint mental effort consistently separate high- and low-performing learners across reading, change–test, and write phases, with cognitive load lower for high performers and joint mental effort showing a “perfect” carry-forward profile. The pattern generalizes across tasks and languages and motivates proactive, phase-aware scaffolding.

Taken together, the studies show that cognitive load plays a central role in programming performance. High load typically leads to longer completion times, more visual regressions, and lower accuracy. Moderate levels of effort are linked to sustained attention and improved outcomes. Code readability and structure influence this balance by lowering unnecessary effort and supporting confidence and focus. Overall, the evidence suggests that well-designed code and adaptive support can regulate cognitive load and improve comprehension efficiency while maintaining engagement.

#### 2.4.1.5 Collaboration and Gaze Patterns in Pair Programming

This theme includes studies on collaborative coding and pair programming, often using Cross Recurrence Quantitative Analysis (CRQA) and eye-tracking to analyze shared gaze patterns and joint attention behaviours. The focus is on understanding how pairs coordinate their problem-solving, synchronize their gaze and attention, and how these interactions affect collaboration quality, communication efficiency, and task success.

Villamor and Rodrigo [20] characterize novice pairs with cross-recurrence plots and related metrics. Successful cases show more flexible, timely shifts and fewer trapped states, while unsuccessful cases display clustered, recurring patterns indicative of lock-in. Case-level differences are clear even when pair-level aggregates are similar.

Furthermore, Tablatin and Rodrigo [36] examine scanpath strategies during debugging and identify signatures that separate high from low performers. High performers adapt strategy to the program, alternating between local, bottom-up reading for complex logic and top-down passes for simpler structures. Low performers show more variable, unstable patterns. The insights translate to navigator–driver guidance on steering joint attention.

Finally, Peterson et al. [43] use visual analytics on C/C++ reading to compare novices and more experienced students. Most gaze transitions are local to nearby lines, with a small set of lines attracting disproportionate dwell. Non-novices make more purposeful long-range transitions consistent with hypothesis checks. These patterns suggest when partners should ground locally and when a navigator can guide longer jumps.

Together, these studies show that effective collaboration relies on flexible and well-timed coordination of visual attention. Productive pairs maintain synchronized yet adaptive gaze patterns that support shared understanding and efficient problem-solving, while repetitive or rigid patterns indicate communication breakdowns. Training and tools that help programmers manage joint attention can therefore enhance collaboration and task performance.

Taken together, the synthesis addresses SIQ7 by showing that the included studies' research questions are closely reflected in how debugging related activity, visual attention, and measurement are operationalized. Across the themes, the tasks, gaze metrics, and performance indicators used in each study correspond directly to the aims the authors seek to investigate, indicating a coherent alignment between conceptual intentions and methodological choices.

### 2.4.2 Methodological Challenges

This subsection addresses SIQ8 by synthesizing the methodological limitations reported across the reviewed studies and identifying constraints that affect the interpretability and comparability of findings. These challenges reflect trade-offs between experimental control and ecological realism, as well as practical constraints in sampling, instrumentation, and task design. Common issues include calibration uncertainty, tool- and participant-related biases, small samples, narrow task scope, and simplified experimental stimuli. Together, these factors influence the reliability, comparability, and generalizability of findings. An overview of all challenges and their corresponding papers is provided in Table 2.4.2.

#### 2.4.2.1 Cognitive load measurement & instrumentation constraints

This theme highlights difficulties in measuring cognitive load accurately within programming tasks. Researchers frequently note uncertainty about whether measured stress or workload stems from the task itself, the novelty of the experimental setup, or unfamiliar instrumentation. Some studies rely on advanced or expensive sensors (e.g., physiological or eye-tracking equipment) that might not generalize well to real-world, scalable contexts. Some also note technical calibration challenges or threshold mismatches across participants. Additionally, the use of students instead of professional programmers introduces potential bias because professionals might exhibit different cognitive responses, attention control, or mental workload patterns.

Fakhoury et al. [57] combined functional Near-Infrared Spectroscopy (fNIRS) and eye tracking to quantify developers' cognitive load during bug localization. They found that lexical inconsistencies increased physiological workload but that self-reported and objective measures of effort did not align, underscoring the difficulty of validating cognitive load metrics. The study also highlighted practical constraints, such as fNIRS calibration demands and signal delays, which complicate synchronization with gaze data.

Sharma and Giannakos [18] used multimodal sensing (eye tracking, electrodermal activity, and heart-rate variability) to infer cognitive and affective states in debugging tasks. Their framework improved the prediction of learner performance but suffered from instrumentation uncertainty and limited scalability beyond controlled environments.

Together, these studies underscore the tension between precision and practicality in cognitive load research. While multimodal physiological instrumentation can yield fine-grained insights into developers' mental effort, the associated technical constraints, calibration sensitivity, and ecological limitations continue to restrict their generalizability beyond controlled experimental contexts.

#### 2.4.2.2 Tooling/setup effects & participant preference bias

This theme captures methodological biases introduced by the tools and environments participants use, especially Integrated Development Environments (IDEs) or specific coding interfaces. Studies under this theme recognize that developer familiarity or preference for certain tools can strongly shape visual attention patterns, fixation behaviour, and efficiency. For instance, a participant comfortable

| Theme | Summary | Included Papers |
|---|---|---|
| **Cognitive load measurement & instrumentation constraints** | Highlights difficulties in reliably measuring cognitive load due to hardware limitations, calibration issues, and mismatches between subjective and physiological indicators. These constraints reduce comparability across studies and complicate the interpretation of workload differences. | [18] [36] [57] |
| **Tooling/setup effects & participant preference bias** | Shows how experimental tools, visual layouts, and unfamiliar setups can influence gaze behaviour, creating confounds and preference biases that may overshadow true cognitive or strategic differences. | [35] [34] |
| **Small samples, short snippets & narrow task scope (internal validity)** | Reflects the widespread use of small participant groups and simplified code snippets, which limit internal validity and restrict insights into realistic, multi-step programming or debugging behaviour. | [26] [55] [44] [46] [52] [53] [48] [22] [41] [28] [45] [59] [29] [30] [60] [54] |
| **Methodological scope & measurement noise** | Points to inconsistencies in metric selection, preprocessing, and analysis pipelines, as well as noise in gaze data that makes cross-study comparison and replication challenging. | [51] [27] [21] [31] [50] [20] [38] [61] [62] [23] [25] [40] [19] [5] |
| **External validity & stimuli choice (generalizability)** | Indicates that most studies rely on short Java or C-like code fragments and student samples, limiting generalizability to real-world programming, diverse languages, and professional developers. | [33] [11] [56] [47] [58] [32] |

**Table 2.4.2:** Overview of challenges and corresponding papers included in the thematic synthesis.

in Visual Studio Code may navigate code differently than one used to Eclipse or IntelliJ. When sample sizes are small, these tooling-related biases become amplified, reducing the reliability of cross-participant comparisons. Researchers note that these preferences create confounds in interpreting whether performance differences reflect actual comprehension ability or tool comfort.

Abid et al. [34] examined how developers' eye movements reveal underlying

mental models during code summarization tasks. The study was conducted in the Eclipse IDE using the iTrace plugin, which maps gaze data to code elements in real time. The authors found that both experts and novices alternated between top-down and bottom-up reading modes depending on navigation behaviour within the IDE. Frequent switching between code, text editors, and other windows reflected how interface layout and navigation options influenced gaze transitions and task pacing.

In a complementary study, Abid et al. [35] replicated earlier eye-tracking research in a more realistic programming environment to reduce ecological bias. By embedding the study directly in Eclipse with scrolling and file-switching enabled, they observed that developers' gaze distributions differed markedly from earlier fixed-window setups. Participants focused less on method signatures and more on method bodies and call terms, suggesting that IDE affordances, such as code context visibility and navigation freedom, substantially alter visual attention patterns.

Together, these studies emphasize that the experimental setup itself can shape how participants engage with source code. Eye-tracking data collected in constrained interfaces may misrepresent natural reading behaviour, while realistic IDE contexts introduce variability linked to individual preferences and familiarity. Accounting for these tooling effects remains critical to ensure that gaze-based findings reflect comprehension processes rather than interface-induced bias.

#### 2.4.2.3 Small samples, short snippets & narrow task scope (internal validity)

The most prevalent limitation category concerns sample size and scope. Most studies employ fewer than 30 participants, typically students from computer science programs. Tasks often use short, simplified code snippets instead of realistic projects, which limits task complexity and cognitive engagement. This reductionist design improves control but sacrifices ecological validity. It is unclear if findings would hold for large, real-world codebases or collaborative settings. The narrow focus also limits task diversity (e.g., only comprehension, no debugging or refactoring), restricting generalization to other programming activities.

Jessup et al. [54] report a pilot comparing experts and novices on long Java files with eye tracking and code perceptions. After exclusions the final sample is 20 participants, and many analyses lack power. The main eye tracking effect is opposite to the hypothesis, with experts showing more fixations, while average fixation duration does not differ. Interactions with reuse intentions are difficult to interpret, given the small cell counts.

Mansoor et al. [52] observe fourteen students working in an IDE on C++ and Python while fixing bugs and adding features. Effort follows the type of task rather than the language. Bug fixing shows longer average and total fixation durations after adjusting for code length. Students often scan nearby lines before they settle on the buggy line. In the feature addition tasks, students who succeed tend to make their first edit earlier. The sample is small and there are several condition contrasts, so the estimates are imprecise and the patterns should be treated as suggestive.

Taken together, findings based on small samples should be treated as sugges-

tive rather than definitive. Larger, pre-registered studies with transparent power analyses and replication would strengthen claims. Expanding task scope and incorporating more realistic programming contexts could also improve generalizability while maintaining experimental control.

#### 2.4.2.4 Methodological scope & measurement noise

This cluster groups studies that question the breadth and robustness of their methodological choices. Many focus on a single programming language (most commonly Java), which limits cross-language comparisons and may bias attention patterns tied to syntax or structure. Others cite individual differences, such as prior experience, coding style, or visual processing habits, that introduce measurement noise in eye-tracking data. Some papers also mention site differences, stimulus inconsistencies (e.g., varying quality of pre-written code summaries), or limited replication efforts (e.g., single-researcher studies). A few note instrument sensitivity, that is, the difficulty of capturing reliable gaze metrics under varying lighting or head-movement conditions.

Kevic et al. [19] examined developers' navigation during change tasks in the Java-based system JabRef using the iTrace plugin in Eclipse. While combining gaze and interaction data increased ecological realism, the single-language setup and differences in participants' tool familiarity introduced potential measurement noise. iTrace also captured gaze data only within the editor, omitting peripheral activity and reducing generalizability to other environments.

Similarly, da Costa et al. [62] analyzed novice programmers reading short Python snippets containing "atoms of confusion." Despite tight experimental control, the study's reliance on Python-only tasks, brief code examples, and manual gaze-point corrections in multi-site sessions introduced procedural variability and instrumentation bias.

Taken together, these studies highlight how methodological scope and setup sensitivity constrain the reliability and generalizability of gaze-based findings. Broader language coverage, standardized calibration procedures, and replication across contexts would reduce measurement noise and strengthen external validity in future work.

#### 2.4.2.5 External validity & stimuli choice (generalizability)

This theme covers the ecological validity problem: how well experimental results translate to real-world programming contexts. Most studies employ controlled, artificial stimuli (e.g., simplified code, fixed-size windows, predefined bug examples) that don't reflect the complexity or unpredictability of professional work. Some also constrain environmental factors, such as constant display sizes or uniform line lengths, that simplify gaze-tracking but limit generalization. As a result, even when internal validity is strong, external validity suffers: the findings might not hold in realistic development environments with multiple files, version control, and team collaboration.

Peitek et al. [32] systematically manipulated the linearity of Java code snippets to examine how structure and comprehension strategy affect programmers' reading order. While their refined stimuli enabled clear comparisons between top-down and bottom-up comprehension, the short, single-file snippets and limited code context

constrained ecological realism, making it uncertain whether gaze behaviours would generalize to larger, multi-class programs.

In contrast, Busjahn and Paterson [47] examined how novices and experts locate and interpret the `main` method when reading short Java programs. Experts typically directed their gaze to `main` almost immediately, treating it as the program's entry point, whereas novices tended to read sequentially from the top of the file. This clear novice–expert contrast was likely accentuated by the single-class, small-scale stimuli and limited ecological realism of the study.

Together, these studies illustrate how experimental control through short, uniform programs clarifies specific comprehension effects but limits the external validity of eye-tracking findings for realistic development settings. While such designs enable precise comparisons across participants and conditions, they capture only a narrow slice of real-world programming activity.

In synthesis, these challenges collectively address SIQ8 by showing that the methodological limitations reported across the studies shape how reliably their findings can be interpreted and compared. Issues such as calibration uncertainty, small samples, narrow task scope, tooling effects, measurement noise, and restricted ecological realism all constrain the validity and generalizability of the evidence base. Together, they clarify where current approaches fall short and where future work must strengthen methodological rigor.

### 2.4.3 Opportunities

This subsection addresses SIQ6 by examining how feedback mechanisms are represented in the reviewed literature, and SIQ8 by identifying opportunities that emerge from the limitations documented in prior work. While earlier work focused on exploratory, small-scale experiments, the papers grouped here identify concrete directions for advancing methodological rigor, expanding participant and task diversity, and scaling data collection toward realistic programming environments. Together, they outline a shift from isolated proof-of-concept studies to cumulative, theory-driven research capable of capturing the social, cognitive, and temporal complexity of programming practice. An overview of all identified opportunities and their corresponding papers is provided in Table 2.4.3.

#### 2.4.3.1 Methodological Refinement & Replication

This theme captures papers that call for improving research design and analytical precision in future studies. Authors emphasize the need to refine measurement frameworks, isolate specific variables, and develop better metrics for code comprehension, readability, and attention. Suggestions often include integrating scanpath analysis, temporal gaze sequence mapping, and debugging-strategy tracking to better understand how attention shifts during code reading. These studies reflect a growing recognition that early eye-tracking research in programming was often exploratory, using descriptive statistics rather than theoretically grounded methods. Future work in this theme focuses on developing standardized methodologies and replicating prior experiments under more controlled yet reproducible conditions. Researchers call for replication to test whether effects such as "expert-novice gaze differences" or "visual layout impacts" persist across different stimuli

| Theme | Summary | Included Papers |
|---|---|---|
| **Methodological Refinement & Replication** | Calls for clearer research methods, better-defined measures, and more consistent analysis. Highlights the need to replicate earlier studies so results can be verified and compared across contexts. | [24] [51] [44] [18] [58] [32] |
| **Collaboration, Pair Programming & Shared Gaze** | Encourages studying how programmers work together using shared gaze and joint attention. Suggests that tracking two people at once can reveal how pairs coordinate and how tools might better support collaboration. | [49] [52] [27] [21] [50] [38] |
| **Language / Paradigm / Task Diversity** | Promotes using more programming languages, task types, and problem contexts. Aims to understand which gaze patterns are universal and which depend on syntax, language style, or task design. | [33] [11] [31] [42] [56] [20] [36] [28] [62] [23] |
| **Participant Diversity, Task Complexity & Richer Metrics** | Calls for including more varied participants and more realistic tasks, along with richer gaze and behavioural measures. Helps capture deeper differences in skill, strategy, and cognitive load. | [53] [57] [45] [43] [26] |
| **Scale-up Studies & Datasets (Larger/-Longitudinal Samples)** | Encourages larger studies, longer-term data collection, and IDE-based tracking to build stronger datasets. Supports more robust analysis and better generalization of findings. | [55] [35] [48] [22] [41] [30] [19] |

**Table 2.4.3:** Overview of opportunities and corresponding papers included in the thematic synthesis.

and participant populations. Overall, this theme represents a maturation of the field, a move from proof-of-concept studies toward methodologically rigorous and replicable science.

Karas et al. [51] refine both operationalization and measurement. They track attention at a semantic level with 19 token categories and analyze ordered scan paths alongside gaze metrics. They map gaze to the abstract syntax tree and compare attention to raw code versus structure, where the two patterns do not always align. The paper documents participant screening and detailed preprocessing, and it releases public data and code to support replication.

Saddler et al. [58] emphasize careful operationalization on real Stack Overflow pages. They define clear AOIs, track which pages participants visits, and use gaze metrics tied to page structure. They show that increasing paragraph counts signif-

icantly reduces fixations on comments, while the number of code blocks does not significantly affect fixation counts. The study provides clear AOI definitions and a replication package, improving reproducibility and comparability across studies.

Peitek et al. [32] offer a replication-oriented study with added methodological control. They introduce a metric for source code linearity and compare fixation sequences with story and execution orders. The findings indicate that snippet linearity has a stronger effect on reading order than experience. Experience effects were only partially replicated, and comprehension strategy plays a minor role. The authors release a replication package and call for careful stimulus control and systematic manipulation of the intended comprehension strategy.

Taken together, these papers push the field toward precise constructs, temporal sequence analyses, controlled and reported stimuli, and open materials so that results can be replicated and trusted across tasks and populations. They mark a shift toward more transparent and theoretically grounded experimentation, where open data and standardized procedures enable meaningful comparison across studies. This growing emphasis on replication and methodological precision strengthens the field's ability to build cumulative knowledge rather than isolated findings.

#### 2.4.3.2 Collaboration, Pair Programming & Shared Gaze

This cluster revolves around extending eye-tracking research beyond individuals to study collaborative and social aspects of programming, such as pair programming, code review, or mentoring scenarios. Papers propose examining joint attention and shared gaze patterns to better understand how teams coordinate, learn, and communicate. The underlying insight is that programming is often a collaborative cognitive activity, yet most current eye-tracking studies observe individuals in isolation. Future research in this direction aims to capture real-world team dynamics by analyzing how developers share focus, negotiate meaning, and guide each other's attention in pair or group settings. Authors also call for multi-user tracking systems, cross-recurrence quantitative analysis (CRQA), and naturalistic observational designs. This theme bridges cognitive science and software engineering collaboration research, highlighting the potential of eye-tracking to model team-level cognition.

Cheng et al. [21] develop a collaborative eye tracking system for synchronous code review and visualize shared gaze in real time. Each code line is defined as an AOI. The interface shows a gaze cursor, line highlights, dwell time shading, and transition lines that connect related code. In a paired user study, reviewers who saw each other's gaze found bugs more efficiently than a control group without gaze visualizations. The paper argues that joint attention cues can coordinate search, reduce redundancy, and support novice reviewers.

Mansoor et al. [52] study students working in a realistic IDE while fixing bugs and adding features in C++ and Python. Bug fixing produces longer average and total fixation duration normalized per character than feature addition. Students often look at adjacent lines before the buggy line, and successful feature addition is associated with an earlier first edit. The study provides a complete replication package, and suggests IDE guidance based on eye movements. These results motivate extending similar gaze-aware cues to collaborative settings such as pair programming and live code review.

Bednarik [50] investigates debugging with eye tracking in an IDE with multiple representations. The study tracks attention across code, output, and visualization, and analyzes how strategies change over time. Novices switch more between views, while experts increasingly integrate code and output. In later phases, greater attention to output and higher switching frequency correlate with finding more bugs. The authors point to gaze-aware cues that coordinate attention across representations and could support mentoring and pair work.

Taken together, these studies show that shared or aligned gaze helps collaborators coordinate search, reduce redundant effort, and find bugs faster while supporting novices. They motivate systems that capture the gaze of more than one developer at the same time and use sequence-based analyses to model joint attention in real tasks, guiding the design of tools for pair programming and code review.

#### 2.4.3.3 Language/Paradigm/Task Diversity

This is one of the broadest clusters, emphasizing the need for greater diversity in programming contexts. Many current studies are limited to a single language (often Java), a single task type (code comprehension), or a single educational population (students). Future research is encouraged to include different programming languages, problem types, and environments to assess the generality of findings. Researchers note that language syntax, paradigm (e.g., procedural vs. object-oriented), and task nature (debugging vs. comprehension) can all affect visual attention and cognitive load. For instance, eye-tracking patterns for Python (whitespace-sensitive syntax) might differ from those for C (bracket-delimited syntax). Similarly, comprehension tasks differ cognitively from debugging or refactoring tasks. Future work here thus aims for comparative studies that reveal which gaze patterns are universal and which are language- or task-dependent. This theme ultimately seeks more external validity and cross-context generalization in eye-tracking research.

de Oliveira et al. [33] examine C snippets that contain language-specific atoms of confusion and compare them to refactored versions without the atoms. In aggregate, atoms increase completion time by about 43.02% percent and raise gaze transitions by about 36.8% percent, while accuracy differences are not statistically significant. Heatmaps show that attention concentrates on the atom regions, indicating that syntax and constructs in imperative code can shift visual effort even when correctness is unchanged.

Obaidellah et al. [11] study algorithmic problem solving with problem statement, example output, and multiple choice answers rather than code reading. They use AOI sequence analysis and scarf plots to classify strategies and identify effective and ineffective solvers. The study shows distinct AOI sequences across time and accuracy groups and calls for follow up work on procedural knowledge and improved stimulus design.

Peitek et al. [23] combine eye tracking with fMRI while participants work on Java snippets under top down and bottom up comprehension and a separate syntax error finding task. The study shows that simultaneous recording is feasible and that adding eye movements gives a finer temporal view of comprehension phases. The MRI setting did lead to some missing eye data and limited vertical precision,

yet the combined signals still revealed effects that fMRI alone could not resolve. The authors share materials and analysis scripts and encourage further combined studies.

Taken together, these papers encourage broader coverage of languages, paradigms, and task types so that we can distinguish universal gaze patterns from those that depend on syntax, representation, and activity.

#### 2.4.3.4 Participant Diversity, Task Complexity & Richer Metrics

This theme deals with diversifying participant populations and increasing task realism and complexity. Papers suggest future studies should recruit participants with varied levels of experience (novices, intermediates, professionals) and design more cognitively demanding tasks that mimic authentic programming work. Early research often relied on homogeneous groups of students and short, simple tasks. Authors here argue that this limits insight into how experience and expertise modulate cognitive strategies. They recommend using richer behavioural and gaze metrics, like pupil dilation, saccade velocity, or time-to-first-fixation. These can be used to capture subtle differences between individuals and contexts. The goal is to link quantitative gaze features more directly to qualitative constructs like expertise, confidence, and problem-solving ability. This theme pushes for multivariate, experience-aware models of programmer cognition.

Andrzejewska and Kotoniak [53] conducted a six-month longitudinal eye-tracking study with first-year computer science students to examine how comprehension skills develop through repeated exposure to programming tasks. They combined behavioural, subjective, and gaze-based measures to track progress across three stages of learning. Shorter fixations and longer saccades in later stages indicated increasing fluency, identifying these metrics as sensitive indicators of growing expertise.

Furthermore, Fakhoury et al. [57] integrated functional Near Infrared Spectroscopy with eye tracking to study cognitive load during bug-localization tasks involving real Java code with lexical and structural inconsistencies. The study showed that oxygenation measures and fixation durations captured distinct dimensions of task difficulty, demonstrating how multimodal data can reveal fine-grained cognitive processes during complex code comprehension.

Ahsan et al. [45] examined how prior programming knowledge differentiates novice programmers' visual behaviour. Using total fixation duration and supervised learning models, they classified performance levels and identified gaze-based patterns predictive of higher task success. These results indicate that even among novices, fixation-based metrics can expose latent expertise differences and inform data-driven models of skill prediction.

Together, these studies reflect a methodological broadening toward richer, experience-aware designs. By combining diverse participant profiles, authentic programming challenges, and multimodal measurement approaches, they demonstrate how advanced gaze and physiological data can better explain individual and contextual variation in cognitive load and performance.

#### 2.4.3.5 Scale-up Studies & Datasets (Larger/Longitudinal Samples)

The final theme calls for scaling up research efforts, both in sample size and temporal duration. Authors propose replicating small-scale studies with larger and more heterogeneous participant groups, conducting longitudinal investigations, and collecting data in realistic programming environments. This reflects awareness that current evidence is drawn mostly from short-term lab experiments. Future work should leverage IDE-integrated eye-tracking tools (e.g., iTrace or Visual Studio plugins) to observe developers in situ over time. Large-scale data collection would allow researchers to apply machine learning models, derive predictive metrics, and establish benchmarks for attention and comprehension. Ultimately, this theme signals a transition toward big-data-driven cognitive modeling in programming research.

Park et al. [55] replicate a previous study on readability rules and provide a complete replication package including stimuli, code, and gaze data. Their approach demonstrates how controlled replications can strengthen methodological reliability and comparability, paving the way for larger shared datasets.

Abid et al. [35] conduct an eye-tracking study within an IDE, observing developers as they summarize real Java methods. The study highlights how gaze data collected in realistic environments can reveal attention patterns linked to developer outputs, and suggests using such setups for broader data collection.

Da Costa and Gheyi [48] conducted three controlled eye-tracking experiments with novice programmers in C, Python, and Java to examine how refactorings and clarifications of confusing code affect comprehension. They report substantial reductions in time, fixations, and regressions for most transformations, though some tasks showed mixed effects. The authors also propose a structured framework for controlled eye-tracking studies and outline the potential for future gaze-informed tools to assist developers during code reading.

Together, these studies illustrate a move toward more open, data-driven, and scalable research in programming cognition. They promote standardized replication practices, IDE-integrated eye-tracking systems, and longitudinal designs capable of capturing how comprehension evolves across tasks, languages, and developer populations.

Taken together, the opportunities identified here address SIQ6 and SIQ8 by highlighting how future work can expand the role of feedback mechanisms and respond directly to the methodological limitations observed in prior studies. The themes point toward more consistent operationalization, greater diversity in tasks and participants, richer and multimodal measurements, and scaled data collection in realistic environments. These directions outline a pathway for developing attention-informed feedback and for strengthening the methodological foundations needed to support cumulative, generalizable research.

## 2.5 Summary

This chapter has presented a systematic review of 48 studies, examined how eye-tracking has been used to investigate programmers' visual attention, cognitive load, and problem-solving behaviour. The findings show that existing research is dominated by short comprehension tasks in Java/C-like languages, with a strong

reliance on fixation-based metrics and traditional inferential analysis. Debugging tasks are comparatively rare, and almost no studies use gaze-based indicators to trigger real-time or adaptive feedback.

Together, the descriptive results and thematic synthesis reveal clear methodological gaps, particularly regarding attention-informed feedback, phase-based cognitive load, and the distinction between proactive and reactive support during debugging.

These insights directly inform the design choices in the next chapter, where the research design for the forthcoming study is outlined.

## 2.6 Limitations

This systematic literature review has several methodological limitations that must be acknowledged. The search was restricted to a single database (Scopus). Relying on only one source, even one as broad as Scopus, raises the risk that relevant studies indexed elsewhere were missed. In addition, the keyword-based search strategy, while carefully constructed, may not have captured all relevant terminology. For example, studies that examined novice debugging behaviours but described them with unconventional terms or synonyms could have been overlooked. Thus, despite efforts to create a comprehensive query, the combination of a single database and constrained keywords may have omitted pertinent research.

The screening and coding processes were conducted mainly by us two, and no formal inter-rater reliability measure was calculated. A shared calibration phase and later consensus discussions were used to align interpretations of the codebook and to handle ambiguous cases, which reduced obvious disagreements. Even with these steps, the absence of a systematic reliability test means that some subjectivity may remain in both the inclusion decisions and how individual variables were coded. The subjective opinion of each person could also change after coding several studies, leaving the gap wider than it started.

This chapter also reflects the literature that was available at the time of the search in September 2025. Studies published after this point may introduce new methods, contexts or findings that change the empirical picture or offer alternative interpretations of novice debugging and attention that are not captured in the present review. These limitations should be considered when interpreting the synthesis, and the findings should be viewed as context dependent and open to refinement in future work.

CHAPTER

# THREE

# DESIGN AND FUTURE PLAN

## 3.1 Introduction

This chapter presents the overall research and technical framework for the experimental study planned in the master's thesis. It builds on the gaps identified in the SLR, particularly the need to examine how real-time, attention-aware feedback can support novice programmers during debugging. The chapter first outlines the research design, including participants, experimental conditions, procedures, measures, and ethical considerations. It then describes the technical system that enables proactive and reactive feedback based on eye-tracking signals. Together, these components form the basis for a controlled within-subject experiment comparing feedback strategies and their effects on cognitive load, attention patterns, and debugging performance. The chapter concludes with a brief plan summarizing the main stages and timeline of the thesis work.

## 3.2 Research Design

### 3.2.1 Introduction

This chapter outlines the research design for the master's thesis that follows this preparatory project. The planned study investigates how different forms of feedback may support novice programmers as they work through debugging tasks under varying cognitive demands. University students with basic programming experience will complete a series of debugging activities while their visual attention is recorded using an eye tracker.

During these tasks, the subjects will encounter two types of support: *proactive feedback*, which is initiated based on indications that the student may be approaching difficulty, and *reactive feedback*, which is delivered only after certain behavioural or eye-tracking signals suggest that support is needed. Because all participants experience both feedback modes, the study can compare their effects within the same individuals.

The primary outcomes of interest include general patterns of attention, broad indicators of debugging performance, and perceptions of workload. The design choices presented in this chapter build on the gaps and opportunities identified

in the systematic literature review in Chapter 2, particularly the need for more research on attention-aware feedback and realistic debugging scenarios involving novice programmers. In particular, the SLR showed that almost no existing studies evaluate feedback that is triggered by real-time gaze signals, and none compare proactive and reactive feedback in novice debugging tasks, which directly motivates the design choices made here.

### 3.2.2 Participants and context

The participants will be university students enrolled in computer science or related programs. All participants will have completed at least one introductory programming course to ensure basic ability to read and edit simple code. Recruitment will be conducted through university student platforms and mailing lists associated with introductory programming courses. A target sample of approximately thirty to fifty students is considered realistic for a lab-based study and aligns with prior eye-tracking research in programming. This population is expected to exhibit a range of debugging strategies while maintaining relatively similar experience levels, making it appropriate for the planned comparisons.

### 3.2.3 Experimental design and feedback conditions

The study uses a within-subject design with two experimental conditions: *proactive feedback* and *reactive feedback*, in addition to an initial control task without feedback. To minimise learning and fatigue effects, the order of the two feedback conditions is counterbalanced across participants.

The SLR identified a lack of studies that use gaze data to activate feedback during debugging, and no studies that systematically compare proactive and reactive mechanisms. The present within-subject setup therefore fills a clear methodological gap by isolating the effects of feedback timing while controlling for individual differences in gaze behaviour.

Within each feedback condition, the system provides matched feedback content so that the primary manipulation is when support is delivered. In the proactive condition, gaze and interaction patterns are monitored by a forecasting agent that triggers hints when the student appears close to being stuck or overloaded. In the reactive condition, hints are triggered only after predefined eye-tracking thresholds are crossed. Figure 3.2.1 visualises the overall structure of the experiment.

### 3.2.4 Procedure

Each session begins with an introduction, informed consent, and eye-tracker calibration. The participant then completes a short pretest assessing initial debugging level, which is later used to classify students into lower- and higher-performing groups for analysis.

Next, the participant completes a baseline debugging task without adaptive feedback, followed by the NASA Task Load Index (NASA TLX) [63] to record perceived workload.

The participant then moves through the two feedback conditions in the counterbalanced order defined in the experimental design. Each condition involves

debugging a new program with either proactive or reactive hints active, and is followed by a NASA TLX rating.

Finally, the session concludes with a brief exit interview. The entire procedure is designed to fit comfortably within a standard laboratory session.

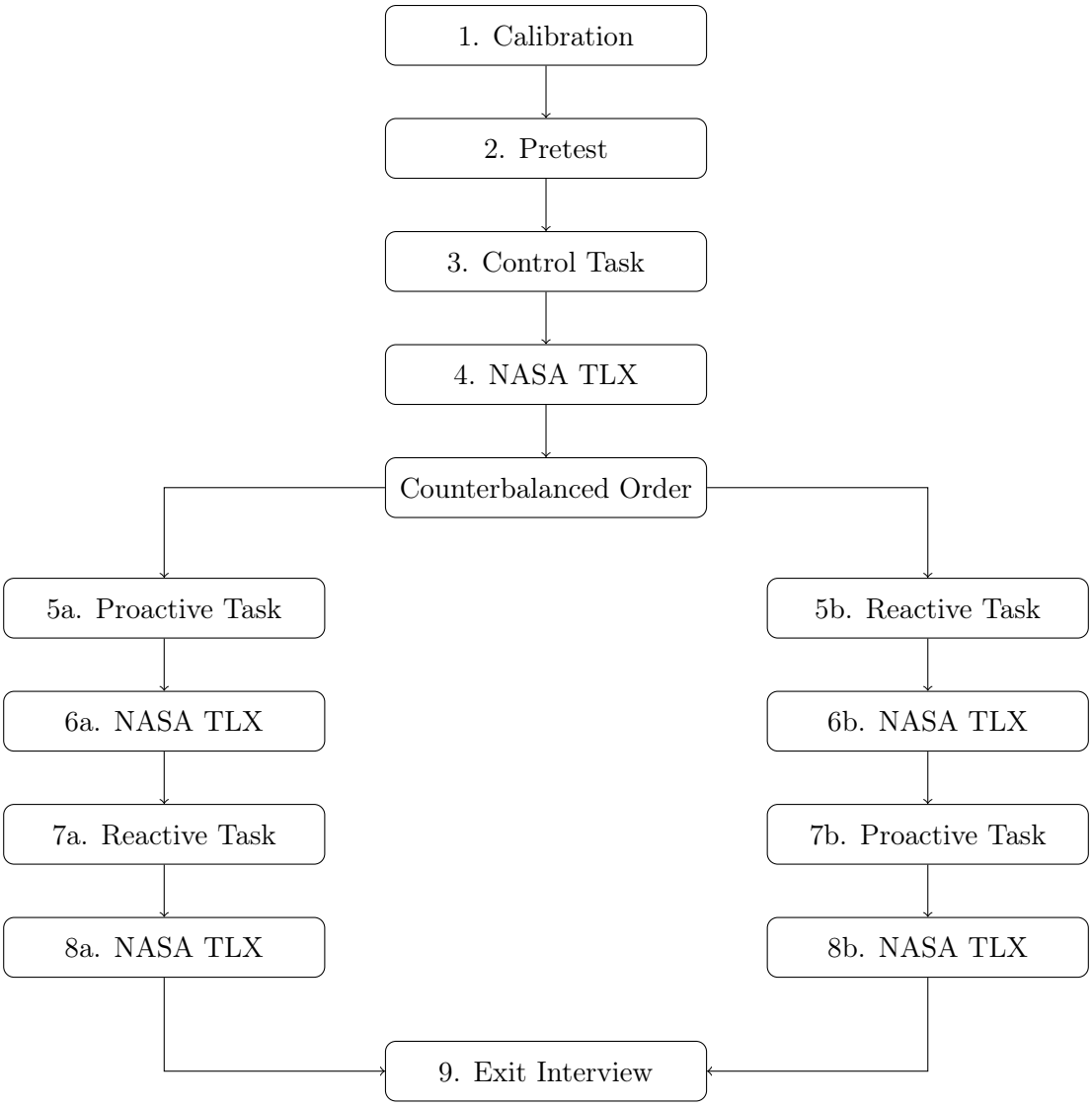



**Figure 3.2.1:** Research design with counterbalanced within-subject feedback conditions.

### 3.2.5 Measures and data collection

The study will draw on a combination of system-generated information, eye-tracking data, and participants' self-reported experiences. Performance will be assessed using general indicators of task progress and success, along with basic information about how participants interacted with the available feedback. Eye-tracking data will be summarised using commonly applied attention metrics to capture how participants visually engaged with different parts of the programming interface. These measures will be used to explore broad patterns of attention and behaviour across the feedback conditions, rather than to make detailed

interpretations of gaze activity.

After each task, participants will complete the NASA Task Load Index, providing an overall impression of workload and perceived demands. Taken together, these sources of information offer a multifaceted view of how students approached the tasks and how their experience varied across conditions.

### 3.2.6 Ethical considerations

All participants will receive written information describing the purpose of the study, the tasks involved, the use of eye-tracking, and their rights as research participants. Participation is voluntary, and students may withdraw at any point without providing a reason and without any consequences for their coursework or academic standing.

All collected data will be stored using pseudonymous identifiers, and the key linking names to these identifiers will be stored separately and securely. Access to raw data will be restricted to the research team, and all publications will present aggregated findings or fully anonymised excerpts from interviews. The tasks involve ordinary programming activities and pose no risk to participants.

Prior to conducting interviews or collecting any data, the project will be submitted to SIKT for ethical assessment. Data collection will only begin after formal approval has been granted.

## 3.3 Technical Design

### 3.3.1 Introduction

This chapter describes the technical design of the prototype system that will support the experimental study. The system connects an external eye-tracking device with a Visual Studio Code (VS Code) extension that delivers proactive and reactive feedback during debugging tasks. The design is lightweight and focuses on the essential components needed to collect data, interpret user state, and present feedback in a controlled laboratory setting.

### 3.3.2 System Overview

The system consists of three main elements: (1) an eye tracker that continuously streams measurements, (2) a VS Code extension that receives these measurements together with the current code context and collects relevant data through the tasks, and (3) a feedback module that decides when to show support to the user. Figure 3.3.1 illustrates how these components interact.

### 3.3.3 Data Inputs and Context

The eye-tracking device provides a small set of standard metrics. These measurements, together with the code problem the subject is working on, form the context used to infer whether the student may need support. The metrics will be processed only at a high level and will not require fine-grained AOI modelling or advanced signal analysis.

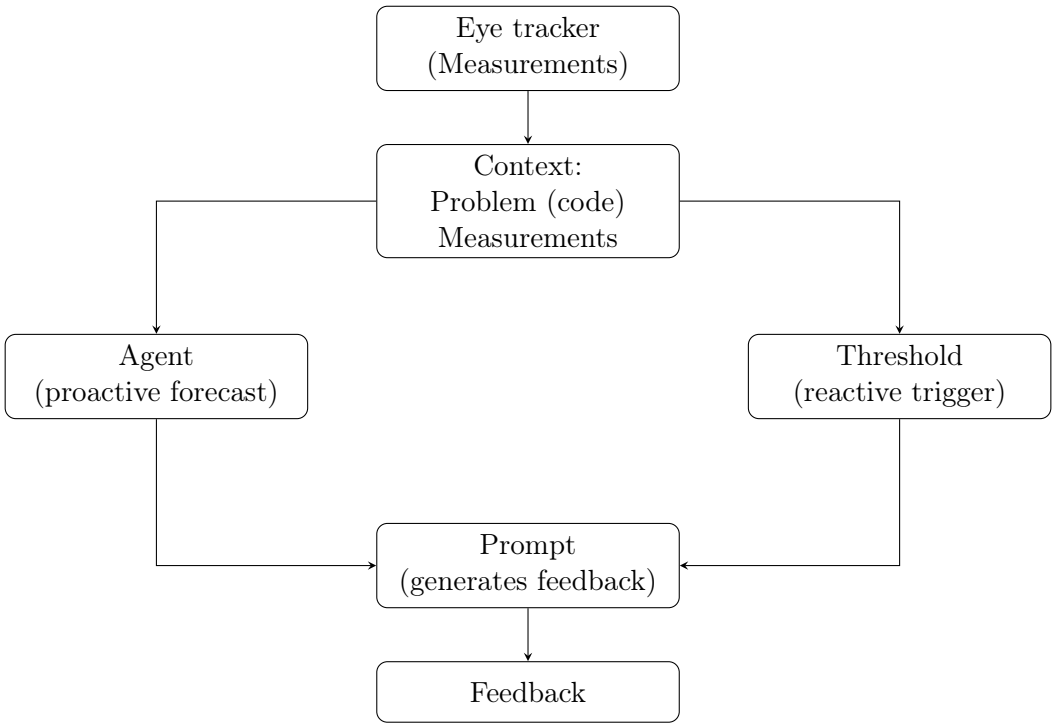



**Figure 3.3.1:** Narrow system flow with proactive (left) and reactive (right) feedback paths.

All eye-tracking data will be continuously stored during the tasks, along with performance-related indicators (to be defined), to support later analysis.

### 3.3.4 Feedback Logic

Two triggering mechanisms will be implemented:

#### 3.3.4.1 Proactive Agent

The proactive feedback will be handled by a trained agent. This model will be developed using existing eye-tracking datasets and will attempt to forecast when the student is approaching difficulty or cognitive overload. When the agent signals a potential need for support, the system generates a prompt.

#### 3.3.4.2 Reactive Threshold

The reactive feedback will use simple rule-based thresholds derived from eye-tracking signals. These thresholds will be defined after pilot observations provide insight into which measurements reliably indicate moments of difficulty. The reactive mechanism only activates once the chosen threshold is crossed.

### 3.3.5 Feedback Generation

When either mechanism triggers, a prompt is generated by a large language model (LLM). The LLM will receive a compact description of the current context (code,

task state, and trigger type) and produce a short feedback message. This feedback is then sent to the VS Code extension.

### 3.3.6 User Interaction w/ Feedback

The VS Code extension displays the feedback prompt to the user. The interaction flow, including accepting or rejecting feedback, temporary disabling, or muting feedback for the remainder of the task, is described in the interaction diagram in Figure 3.3.2. The extension also records which feedback events occurred and how the participant responded.

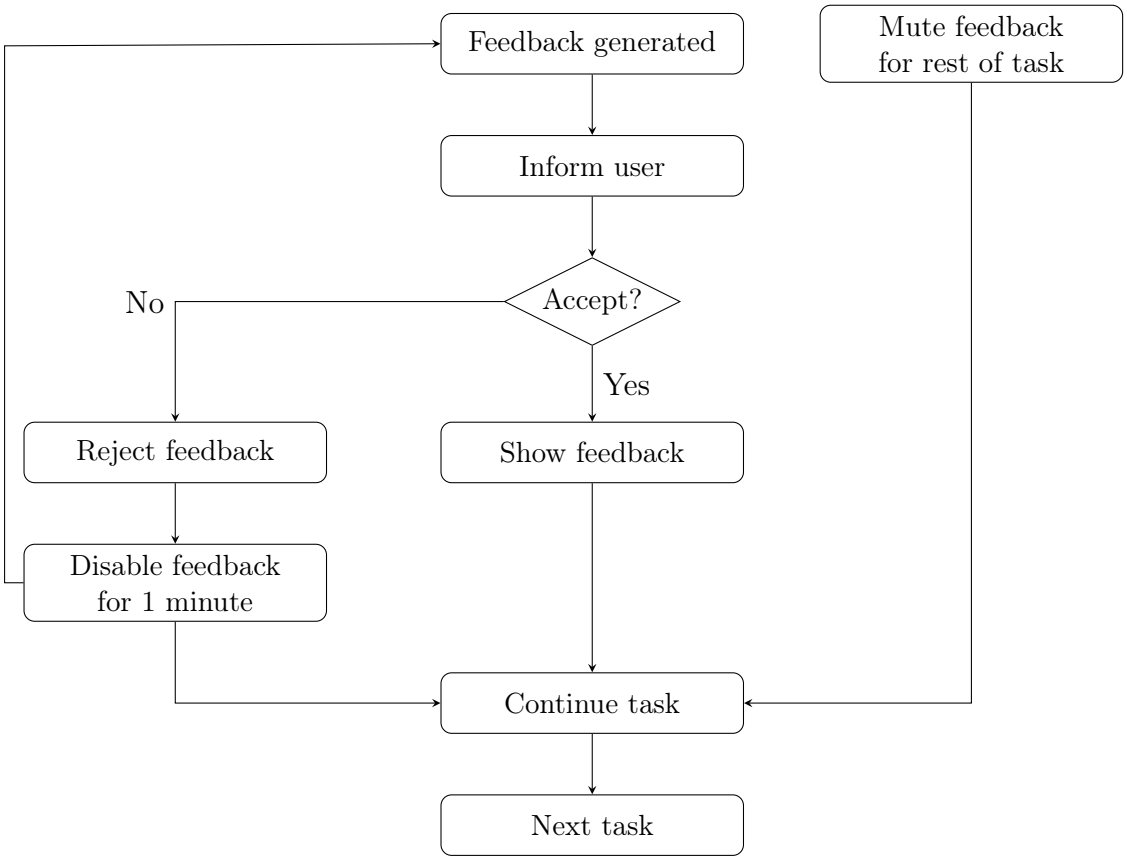



**Figure 3.3.2:** Interaction flow when feedback is generated during a task. Users may accept or reject feedback, temporarily disable it, or mute it for the remainder of the current task.

Figure 3.3.3 shows an early Figma prototype of the feedback prompt used in the study. In this design, the system does not immediately interrupt the user with a full feedback message. Instead, it first presents a lightweight notification that informs the user that feedback is available. This follows the interaction flow in Figure 3.3.2, where the user must choose whether to receive support or to mute the assistant for a short period. Presenting feedback in two steps, first as a request and then as the actual message, helps reduce unnecessary interruptions and ensures that feedback is delivered only when the user indicates readiness, as shown in Figure 3.3.4. This also supports the study's need for controlled exposure, since the acceptance decision determines whether the upcoming message is registered

as delivered feedback. All prototypes that illustrate how the message and mute options appear in the interface are available in Appendix B.1 to B.4.

Note that these prototypes are in an early stage and may need further adjustments after further testing.

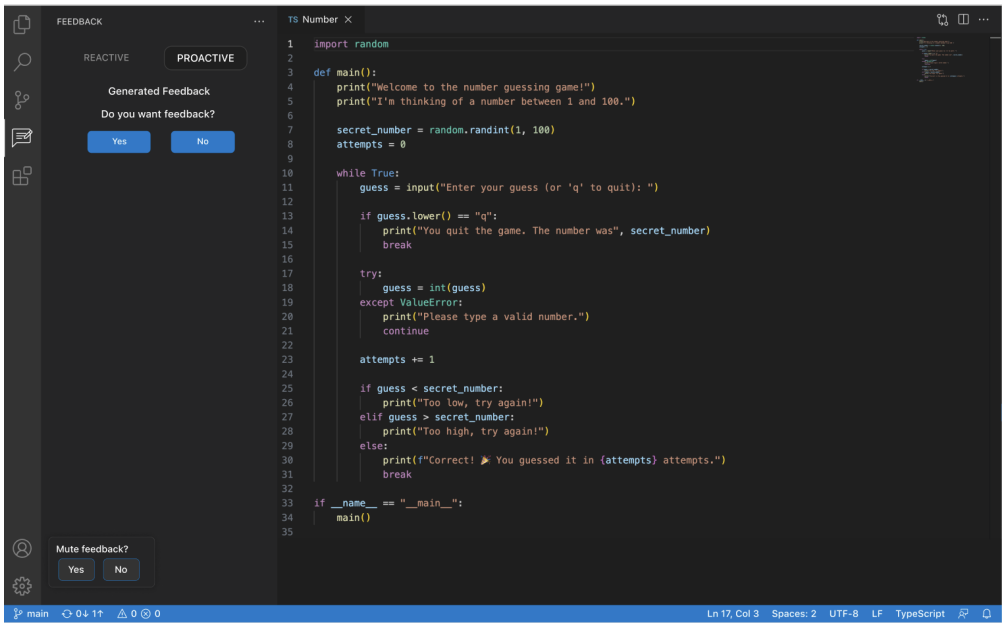



**Figure 3.3.3:** Notification informing the user that feedback is available

### 3.3.7 Data Storage

Throughout the session, the system stores eye tracking measurements as a continuous stream, timestamps of feedback triggers, user responses to feedback, and performance indicators such as completion time or task success. These data sources will support later comparison of proactive and reactive conditions. The technical decisions about data storage are not finalized yet, but the current plan is to use a lightweight and structured format that allows reliable synchronization between eye tracking data, interaction logs, and task performance. The final choice will depend on the integration requirements of the VS Code extension and the analysis pipeline, and may be adjusted as implementation and pilot testing reveal practical constraints.

### 3.3.8 Summary

The system integrates eye tracking signals, a VS Code extension, and a feedback generation module to deliver proactive and reactive support during debugging tasks. The design focuses on simplicity and on providing the essential technical functionality required for the controlled experimental study. The eye tracker supplies continuous measurements that, together with the current code context, form the basis for detecting moments where support may be beneficial. These signals are processed through both a trained proactive agent and a rule-based reactive threshold, which trigger a prompt when predefined conditions are met.

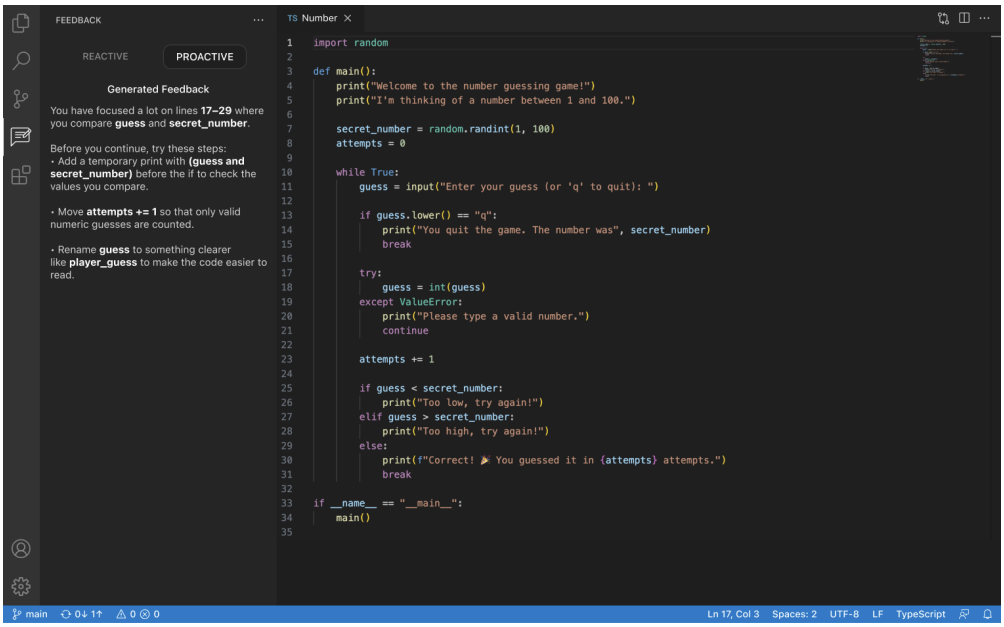



**Figure 3.3.4:** Prototype showing how accepted feedback is presented to the user in VS Code

Once triggered, the system generates a short context-aware feedback message and presents it through the VS Code extension. The user interaction flow ensures that participants maintain control over when feedback is shown, allowing them to accept, reject, delay, or mute assistance during the task. All events, measurements, and responses are logged for later comparison of the two feedback conditions. The technical design therefore provides a coherent framework that supports data collection, user-facing interaction, and controlled delivery of feedback during the experimental tasks.

## 3.4 Future Plan

This section outlines the preliminary plan for the master's thesis that follows this preparatory project. The timeline spans 21 weeks and is structured as a waterfall-style process informed by the findings of the SLR. Each stage builds on the previous one, moving from design refinement to system implementation, pilot testing, data collection, analysis, and final writing. The plan provides a clear overview of the intended workflow but may be adjusted as practical constraints, pilot results, or supervisory feedback require. The tentative plan is visualized in Table 3.4.1.

| **Stage** | **Description** | **Weeks** |
|---|---|---|
| Refinement of Research Design | Finalise task selection, feedback logic, measurement strategy, and analysis plan based on SLR findings and supervisor input. | 1–2 |
| System Development | Implement core system components: VS Code extension, feedback triggering logic, logging modules, and initial proactive agent. | 3–7 |
| Pilot Testing | Conduct small-scale pilot (3–5 participants), refine thresholds, test instructions, and validate system stability. | 8–9 |
| Easter Holiday | | 10 |
| Main Data Collection | Recruit participants and run the full experiment using the counterbalanced within-subject design. | 11–13 |
| Data Processing and Analysis | Clean and align eye-tracking streams, logs, and performance data; run statistical comparisons across conditions. | 14–17 |
| Writing and Synthesis | Write and revise thesis chapters, integrate findings with literature, finalise figures, tables, and appendices. | 18–21 |

**Table 3.4.1:** Tentative timeline and stages for the master's thesis project.

# REFERENCES


[1] Sue Fitzgerald et al. "Debugging: finding, fixing and flailing, a multi-institutional study of novice debuggers". In: *Computer Science Education* 18.2 (June 2008). Publisher: Routledge _eprint: https://doi.org/10.1080/08993400802114508, pp. 93–116. ISSN: 0899-3408. DOI: 10.1080/08993400802114508. URL: https://doi.org/10.1080/08993400802114508 (visited on 11/11/2025).

[2] Beth Simon et al. "Common sense computing (episode 4): debugging". In: *Computer Science Education* 18.2 (June 2008). Publisher: Routledge _eprint: https://doi.org/10.1080/08993400802114698, pp. 117–133. ISSN: 0899-3408. DOI: 10.1080/08993400802114698. URL: https://doi.org/10.1080/08993400802114698 (visited on 11/11/2025).

[3] Billy Javier. "Understanding their Voices from Within: Difficulties and Code Comprehension of Life-Long Novice Programmers". en. In: *International Journal of Arts, Sciences and Education* 1.1 (July 2021), pp. 53–73. ISSN: 2799-1091. URL: https://www.mail.ijase.org/index.php/ijase/article/view/1 (visited on 11/11/2025).

[4] Pablo Romero et al. "Debugging strategies and tactics in a multi-representation software environment". In: *International Journal of Human-Computer Studies* 65.12 (Dec. 2007), pp. 992–1009. ISSN: 1071-5819. DOI: 10.1016/j.ijhcs.2007.07.005. URL: https://www.sciencedirect.com/science/article/pii/S1071581907001000 (visited on 11/11/2025).

[5] Y.-T. Lin et al. "Tracking Students' Cognitive Processes during Program Debugging-An Eye-Movement Approach". English. In: *IEEE Transactions on Education* 59.3 (2016). Publisher: Institute of Electrical and Electronics Engineers Inc., pp. 175–186. ISSN: 15579638 (ISSN); 00189359 (ISSN). DOI: 10.1109/TE.2015.2487341. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84951887152&doi=10.1109%2FTE.2015.2487341&partnerID=40&md5=11da854b822ea8bb10f58dcda21fbfeb.

[6] Jacqueline Whalley, Amber Settle, and Andrew Luxton-Reilly. "A Think-Aloud Study of Novice Debugging". en. In: *ACM Transactions on Computing Education* 23.2 (June 2023), pp. 1–38. ISSN: 1946-6226, 1946-6226. DOI: 10.1145/3589004. URL: https://dl.acm.org/doi/10.1145/3589004 (visited on 11/11/2025).

[7] Rolf-Helge Pfeiffer. "The Impact of Continuous Code Quality Assessment on Defects". In: *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. ISSN: 2576-3148. Sept. 2021, pp. 624–628. DOI: 10.1109/ICSME52107.2021.00069. URL: https://ieeexplore.ieee.org/abstract/document/9609105 (visited on 11/11/2025).

[8] Tamara van Gog et al. "Uncovering cognitive processes: Different techniques that can contribute to cognitive load research and instruction". In: *Computers in Human Behavior*. Including the Special Issue: State of the Art Research into Cognitive Load Theory 25.2 (Mar. 2009), pp. 325–331. ISSN: 0747-5632. DOI: 10.1016/j.chb.2008.12.021. URL: https://www.sciencedirect.com/science/article/pii/S0747563208002318 (visited on 11/10/2025).

[9] Jennifer L Branch. "Investigating the Information-Seeking Processes of Adolescents: The Value of Using Think Alouds and Think Afters". In: *Library & Information Science Research* 22.4 (Nov. 2000), pp. 371–392. ISSN: 0740-8188. DOI: 10.1016/S0740-8188(00)00051-7. URL: https://www.sciencedirect.com/science/article/pii/S0740818800000517 (visited on 11/10/2025).

[10] Unaizah Obaidellah, Mohammed Al Haek, and Peter C.-H. Cheng. "A survey on the usage of eye-Tracking in computer programming". English. In: *ACM Computing Surveys* 51.1 (2019). Publisher: Association for Computing Machinery. ISSN: 0360-0300. DOI: 10.1145/3145904.

[11] U. Obaidellah, M. Raschke, and T. Blascheck. "Classification of strategies for solving programming problems using AoI sequence analysis". English. In: ed. by S.N. Spencer. Association for Computing Machinery acmhelp@acm.org, 2019. ISBN: 9781450357067 (ISBN); 1595933050 (ISBN); 9781450341257 (ISBN); 9781595939821 (ISBN); 9781605589947 (ISBN); 9781450312257 (ISBN); 9781450327510 (ISBN); 9781450367097 (ISBN); 9781595933058 (ISBN). DOI: 10.1145/3314111.3319825. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85069432028&doi=10.1145%2F3314111.3319825&partnerID=40&md5=8c02b28294a75ce234846afca8da2b9d.

[12] John Hattie. *Visible learning: A synthesis of over 800 meta-analyses relating to achievement*. English. Routledge Taylor & Francis Group, 2008. ISBN: 978-0-203-88733-2. DOI: 10.4324/9780203887332.

[13] Andrew Emerson et al. "Predicting early and often: Predictive student modeling for block-based programming environments". English. In: *EDM - Proc. Int. Conf. Educ. Data Min.* Journal Abbreviation: EDM - Proc. Int. Conf. Educ. Data Min. International Educational Data Mining Society, 2019, pp. 39–48. ISBN: 978-1-7336736-0-0.

[14] Albert T. Corbett and John R. Anderson. "Locus of feedback control in computer-based tutoring: Impact on learning rate, achievement and attitudes". English. In: *Conf Hum Fact Comput Syst Proc.* Journal Abbreviation: Conf Hum Fact Comput Syst Proc. 2001, pp. 245–252. ISBN: 978-1-58113-327-1. DOI: 10.1145/365024.365111.

[15] Vincent Aleven et al. "Toward Meta-cognitive Tutoring: A Model of Help Seeking with a Cognitive Tutor". EN. In: *International Journal of Artificial Intelligence in Education* 16.2 (May 2006). Publisher: SAGE Publications, pp. 101–128. ISSN: 1560-4292. DOI: 10.3233/IRG-2006-16(2)02. URL: https://journals.sagepub.com/action/showAbstract (visited on 11/26/2025).

[16] Barbara Kitchenham and Stuart M. Charter. *(PDF) Guidelines for performing Systematic Literature Reviews in Software Engineering*. en. 2007. URL: https://www.researchgate.net/publication/302924724_Guidelines_for_performing_Systematic_Literature_Reviews_in_Software_Engineering (visited on 10/20/2025).

[17] Forrest Shull, Janice Singer, and Dag I. K. Sjøberg, eds. *Guide to advanced empirical software engineering*. en. London: Springer, 2008. ISBN: 978-1-84800-043-8 978-1-84800-044-5.

[18] K. Sharma and M. Giannakos. "Carry-forward effect: providing proactive scaffolding to learning processes". English. In: *Behaviour and Information Technology* 44.11 (2025). Publisher: Taylor and Francis Ltd., pp. 2760–2799. ISSN: 0144929X (ISSN); 13623001 (ISSN). DOI: 10.1080/0144929X.2024.2411592. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85206mes578492&doi=10.1080%2F0144929X.2024.2411592&partnerID=40&md5=fd449b649e3378b0049787e8f035cded.

[19] K. Kevic et al. "Tracing software developers' eyes and interactions for change tasks". English. In: Association for Computing Machinery, Inc acmhelp@acm.org, 2015, pp. 202–213. ISBN: 9781450336758 (ISBN). DOI: 10.1145/2786805.2786864. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84960351241&doi=10.1145%2F2786805.2786864&partnerID=40&md5=98ebbcd332a5848dc9631ef2dded709d.

[20] M. Villamor and M.M. Rodrigo. "Gaze collaboration patterns of successful and unsuccessful programming pairs". English. In: ed. by M.M.T. Rodrigo et al. Asia-Pacific Society for Computers in Education No. 300, Jhongda Road, Jhongli City, Taoyuan County 32001, 2018, pp. 113–122. ISBN: 9789869401289 (ISBN). URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85060061224&partnerID=40&md5=d4e3cce51c1b36995b1a89e97669cd95.

[21] S. Cheng et al. "Collaborative eye tracking based code review through real-time shared gaze visualization". English. In: *Frontiers of Computer Science* 16.3 (2022). Publisher: Higher Education Press Limited Company. ISSN: 20952236 (ISSN); 20952228 (ISSN). DOI: 10.1007/s11704-020-0422-1. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85119440560&doi=10.1007%2Fs11704-020-0422-1&partnerID=40&md5=34deccb67a318bfb31bb0a9dfc050dfc.

[22] S. Yabesi et al. "Exploring the Effects of Urgency and Reputation in Code Review An Eye-Tracking Study". English. In: IEEE Computer Society, 2024, pp. 202–213. ISBN: 26437147 (ISSN); 26437171 (ISSN); 9781450392983 (ISBN); 9798400705861 (ISBN); 9798331502232 (ISBN). DOI: 10.1145/3643916.3644425. URL: https://www.scopus.com/inward/record.uri?eid=2-

s2.0-85197289403&doi=10.1145%2F3643916.3644425&partnerID=40&md5=208231758c7a5687498e7b1f5c38bc82.

[23] N. Peitek et al. "Simultaneous measurement of program comprehension with fMRI and eye tracking: A case study". English. In: IEEE Computer Society help@computer.org, 2018. ISBN: 19493770 (ISSN); 19493789 (ISSN); 9781665452236 (ISBN); 9798400710476 (ISBN); 9781728129686 (ISBN); 9781467378994 (ISBN); 9781450344272 (ISBN); 9781450375801 (ISBN); 9781450386654 (ISBN); 9781450358231 (ISBN); 9781509040391 (ISBN); 9781450394277 (ISBN). DOI: 10.1145/3239235.3240495. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85061492623&doi=10.1145%2F3239235.3240495&partnerID=40&md5=3f1ec979c649b54fef6695435efcea73.

[24] C.A. Pacol, M.M.T. Rodrigo, and C.L.S. Tablatin. "A Comparative Study of High and Low Performing Students' Visual Effort and Attention When Identifying Syntax Errors". English. In: ed. by D.D. Schmorrow and C.M. Fidopiastis. Vol. 14694 LNAI. Springer Science and Business Media Deutschland GmbH, 2024, pp. 77–94. ISBN: 16113349 (ISSN); 03029743 (ISSN); 9789819698936 (ISBN); 9789819698042 (ISBN); 9789819698110 (ISBN); 9789819698905 (ISBN); 9789819512324 (ISBN); 9783032026019 (ISBN); 9783032008909 (ISBN); 9783031915802 (ISBN); 9789819698141 (ISBN); 9783031984136 (ISBN). DOI: 10.1007/978-3-031-61569-6_6. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85196770981&doi=10.1007%2F978-3-031-61569-6_6&partnerID=40&md5=8fd18b086a815cdfa0f1dce8425c0597.

[25] T.R. Beelders and Jean-Pierre L. Du Plessis. "Syntax highlighting as an influencing factor when reading and comprehending source code". English. In: *Journal of Eye Movement Research* 9.1 (2016). Publisher: International Group for Eye Movement Research, pp. 2207–2219. ISSN: 1995-8692.

[26] J. Katona. "An Eye Movement Study in Unconventional Usage of Different Software Tools". English. In: *Sensors* 23.8 (2023). Publisher: MDPI. ISSN: 14248220 (ISSN). DOI: 10.3390/s23083823. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85153942894&doi=10.3390%2Fs23083823&partnerID=40&md5=6001500582c6bcc194d1c31636f722f8.

[27] S. Papavlasopoulou, K. Sharma, and M.N. Giannakos. "Coding activities for children: Coupling eye-tracking with qualitative data to investigate gender differences". English. In: *Computers in Human Behavior* 105 (2020). Publisher: Elsevier Ltd. ISSN: 07475632 (ISSN). DOI: 10.1016/j.chb.2019.03.003. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85077680900&doi=10.1016%2Fj.chb.2019.03.003&partnerID=40&md5=61b25b2b692fa17d04f5a85042dd4c8b.

[28] H.G. Lapierre, P. Charland, and P.-M. Léger. "Looking "Under the hood" of learning computer programming: the emotional and cognitive differences between novices and beginners". English. In: *Computer Science Education* 34.3 (2024). Publisher: Routledge, pp. 331–352. ISSN: 08993408 (ISSN); 17445175 (ISSN). DOI: 10.1080/08993408.2023.2214033. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85160612722&doi=10.1080%2F08993408.2023.2214033&partnerID=40&md5=6b21f3a63562f537565ea583f9d7ba08.

[29] S.D. Aljehane, B. Sharif, and J.I. Maletic. "Studying Developer Eye Movements to Measure Cognitive Workload and Visual Effort for Expertise Assessment". English. In: *Proceedings of the ACM on Human-Computer Interaction* 7.ETRA (2023). Publisher: Association for Computing Machinery. ISSN: 25730142 (ISSN). DOI: 10.1145/3591135. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85160411121&doi=10.1145%2F3591135&partnerID=40&md5=60c30fbfd016a6f982ec1f02b50baf29.

[30] D. Binkley et al. "The impact of identifier style on effort and comprehension". English. In: *Empirical Software Engineering* 18.2 (2013). Publisher: Kluwer Academic Publishers, pp. 219–276. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-012-9201-4. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84880698340&doi=10.1007%2Fs10664-012-9201-4&partnerID=40&md5=cdff8284430de114e2cc53194c5ee0ca.

[31] J.A.S. da Costa et al. "Evaluating refactorings for disciplining #ifdef annotations: An eye tracking study with novices". English. In: *Empirical Software Engineering* 26.5 (2021). Publisher: Springer. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-021-10002-8. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85109775957&doi=10.1007%2Fs10664-021-10002-8&partnerID=40&md5=8d85db8d17440759ddb7e9901ac146c

[32] N. Peitek, J. Siegmund, and S. Apel. "What drives the reading order of programmers? an eye tracking study". English. In: IEEE Computer Society help@computer.org, 2020, pp. 342–353. ISBN: 9781450379588 (ISBN). DOI: 10.1145/3387904.3389279. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85091947378&doi=10.1145%2F3387904.3389279&partnerID=40&md5=d19c6e8c5559cd760effef95b873f969.

[33] B. de Oliveira et al. "Atoms of Confusion: The Eyes Do Not Lie". English. In: Association for Computing Machinery, 2020, pp. 243–252. ISBN: 9781450385855 (ISBN); 9781450314398 (ISBN); 9781450396387 (ISBN); 9781450390019 (ISBN); 9781450390217 (ISBN); 9781450348270 (ISBN); 9781450381963 (ISBN); 9781450322485 (ISBN); 9781450348201 (ISBN); 9781450364454 (ISBN). DOI: 10.1145/3422392.3422437. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85099406691&doi=10.1145%2F3422392.3422437&partnerID=40&md5=825d6e56c7d865c31038517dc2705019.

[34] N.J. Abid, J.I. Maletic, and B. Sharif. "Using developer eye movements to externalize the mental model used in code summarization tasks". English. In: ed. by S.N. Spencer. Association for Computing Machinery acmhelp@acm.org, 2019. ISBN: 9781450357067 (ISBN); 1595933050 (ISBN); 9781450341257 (ISBN); 9781595939821 (ISBN); 9781605589947 (ISBN); 9781450312257 (ISBN); 9781450327510 (ISBN); 9781450367097 (ISBN); 9781595933058 (ISBN). DOI: 10.1145/3314111.3319834. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85069520523&doi=10.1145%2F3314111.3319834&partnerID=40&md5=7a1d867eab81ff3d871f6f2c5a29598c.

[35] N.J. Abid et al. "Developer Reading Behavior while Summarizing Java Methods: Size and Context Matters". English. In: vol. 2019-May. IEEE Computer Society help@computer.org, 2019, pp. 384–395. ISBN: 02705257 (ISSN); 9781450392266 (ISBN); 9780769528625 (ISBN); 9781450305761 (ISBN); 9781450305839 (ISBN);

1581132069 (ISBN); 9781450357401 (ISBN); 9798400705007 (ISBN); 9781605589701 (ISBN); 9781595934055 (ISBN); 9781595934079 (ISBN). DOI: 10.1109/ICSE.2019.00052. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85069452233&doi=10.1109%2FICSE.2019.00052&partnerID=40&md5=3684711fa0acf41456d56af452c39f6c.

[36] C.L.S. Tablatin and M.M.T. Rodrigo. "Identifying Code Reading Strategies in Debugging using STA with a Tolerance Algorithm". English. In: *APSIPA Transactions on Signal and Information Processing* 11.1 (2022). Publisher: Now Publishers Inc. ISSN: 20487703 (ISSN). DOI: 10.1561/116.00000040. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85126395405&doi=10.1561%2F116.00000040&partnerID=40&md5=657f8af0371fa346f1928743c4ce9075.

[37] M.M. Villamor and M.M.T. Rodrigo. "Predicting Pair Success in a Pair Programming Eye Tracking Experiment Using Cross-Recurrence Quantification Analysis". English. In: *APSIPA Transactions on Signal and Information Processing* 11.1 (2022). Publisher: Now Publishers Inc. ISSN: 20487703 (ISSN). DOI: 10.1561/116.00000031. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85129478282&doi=10.1561%2F116.00000031&partnerID=40&md5=0ddbe4c872daf226fd15ad12f9e6a783.

[38] A. Jbara and D.G. Feitelson. "How programmers read regular code: a controlled experiment using eye tracking". English. In: *Empirical Software Engineering* 22.3 (2017). Publisher: Springer New York LLC barbara.b.bertram@gsk.com, pp. 1440–1477. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-016-9477-x. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-84996538088&doi=10.1007%2Fs10664-016-9477-x&partnerID=40&md5=aa6f708dc628bf15fe31c4aa73a45921.

[39] Marthie Schoeman. "Reading skills can predict the-programming performance of novices: An eye-Tracking study". English. In: *Perspectives in Education* 37.2 (2019). Publisher: University of Pretoria, pp. 35–52. ISSN: 0258-2236. DOI: 10.18820/2519593X/pie.v37i2.3.

[40] J. Prather et al. "The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers". English. In: vol. 1. Association for Computing Machinery, Inc, 2024, pp. 469–486. ISBN: 9798400704765 (ISBN); 9798400704758 (ISBN). DOI: 10.1145/3632620.3671116. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85202820894&doi=10.1145%2F3632620.3671116&partnerID=40&md5=9ac7d21656a62c199aa2ab5222555240.

[41] N. Al Madi et al. "From Novice to Expert: Analysis of Token Level Effects in a Longitudinal Eye Tracking Study". English. In: vol. 2021-May. IEEE Computer Society, 2021, pp. 172–183. ISBN: 9781665414036 (ISBN). DOI: 10.1109/ICPC52881.2021.00025. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85107535745&doi=10.1109%2FICPC52881.2021.00025&partnerID=40&md5=fbe184ec4deca649d7c44fdb70a16f33.

[42] G. Cheng et al. "Exploring differences in self-regulated learning strategy use between high- and low-performing students in introductory programming: An analysis of eye-tracking and retrospective think-aloud data from program comprehension". English. In: *Computers and Education* 208 (2024).

Publisher: Elsevier Ltd. ISSN: 03601315 (ISSN). DOI: 10.1016/j.compedu.2023.104948. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85174445489&doi=10.1016%2Fj.compedu.2023.104948&partnerID=40&md5=b7047d99b297ec5909d217d05e3c5ceb.

[43] C.S. Peterson et al. "Visually analyzing students' gaze on C++ code snippets". English. In: Institute of Electrical and Electronics Engineers Inc., 2019, pp. 18–25. ISBN: 9781728122434 (ISBN). DOI: 10.1109/EMIP.2019.00011. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85069506905&doi=10.1109%2FEMIP.2019.00011&partnerID=40&md5=cc0250aca366d93a39246b3f61e00fd9.

[44] S. Matsumoto et al. "Analyzing reading pattern of simple c source code consisting of only assignment and arithmetic operations based on data dependency relationship by using eye movement". English. In: ed. by H. Mori and S. Yamamoto. Vol. 10904 LNCS. Springer Verlag service@springer.de, 2018, pp. 545–561. ISBN: 16113349 (ISSN); 03029743 (ISSN); 9789819698936 (ISBN); 9789819698042 (ISBN); 9789819698110 (ISBN); 9789819698905 (ISBN); 9789819512324 (ISBN); 9783032026019 (ISBN); 9783032008909 (ISBN); 9783031915802 (ISBN); 9789819698141 (ISBN); 9783031984136 (ISBN). DOI: 10.1007/978-3-319-92043-6_44. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85050402674&doi=10.1007%2F978-3-319-92043-6_44&partnerID=40&md5=85f635422af246f7b913a183a2239ef8.

[45] Z. Ahsan and U. Obaidellah. "Predicting expertise among novice programmers with prior knowledge on programming tasks". English. In: Institute of Electrical and Electronics Engineers Inc., 2020, pp. 1008–1016. ISBN: 9789881476883 (ISBN). URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85100945995&partnerID=40&md5=eb461fc88f3277273a90880d034add8

[46] M. Segedinac et al. "Assessing code readability in Python programming courses using eye-tracking". English. In: *Computer Applications in Engineering Education* 32.1 (2024). Publisher: John Wiley and Sons Inc. ISSN: 10613773 (ISSN); 10990542 (ISSN). DOI: 10.1002/cae.22685. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85173801740&doi=10.1002%2Fcae.22685&partnerID=40&md5=b3a46ea311061c18ac36ba47cac0e936.

[47] T. Busjahn and J.H. Paterson. "Looking at the main Method - An Educator's Perspective". English. In: Association for Computing Machinery, 2021. ISBN: 9781450385855 (ISBN); 9781450314398 (ISBN); 9781450396387 (ISBN); 9781450390019 (ISBN); 9781450390217 (ISBN); 9781450348270 (ISBN); 9781450381963 (ISBN); 9781450322485 (ISBN); 9781450348201 (ISBN); 9781450364454 (ISBN). DOI: 10.1145/3488042.3488068. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85119874175&doi=10.1145%2F3488042.3488068&partnerID=40&md5=e7a0c2563d13dfe07a39637d26d57cdc.

[48] J.A. da Costa and R. Gheyi. "Evaluating the Code Comprehension of Novices with Eye Tracking". English. In: Association for Computing Machinery, 2023, pp. 332–341. ISBN: 9798400707865 (ISBN); 9798400709401 (ISBN); 9781450399883 (ISBN); 9798400708206 (ISBN); 9798400708213 (ISBN); 9798400708862 (ISBN); 9798400707629 (ISBN). DOI: 10.1145/3629479.3629490. URL: https:

//www.scopus.com/inward/record.uri?eid=2-s2.0-85180153244&doi=10.1145%2F3629479.3629490&partnerID=40&md5=16e5c986860e3759dc56e1112b9af275.

[49] R. Talsma, E. Barendsen, and S. Smetsers. "Analyzing the influence of block highlighting on beginning programmers' reading behavior using eye tracking". English. In: ed. by D. Stikkolorum and E. Rahimi. Association for Computing Machinery, 2020. ISBN: 9781450385855 (ISBN); 9781450314398 (ISBN); 9781450396387 (ISBN); 9781450390019 (ISBN); 9781450390217 (ISBN); 9781450348270 (ISBN); 9781450381963 (ISBN); 9781450322485 (ISBN); 9781450348201 (ISBN); 9781450364454 (ISBN). DOI: 10.1145/3442481.3442505. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85103598036&doi=10.1145%2F3442481.3442505&partnerID=40&md5=a684b3eb2da00934b2a26b4c1cd8f640.

[50] R. Bednarik. "Expertise-dependent visual attention strategies develop over time during debugging with multiple code representations". English. In: *International Journal of Human Computer Studies* 70.2 (2012), pp. 143–155. ISSN: 10715819 (ISSN); 10959300 (ISSN). DOI: 10.1016/j.ijhcs.2011.09.003. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-80855156566&doi=10.1016%2Fj.ijhcs.2011.09.003&partnerID=40&md5=a637c3c4acb3acfd4bada8bffe80cc2d.

[51] Z. Karas et al. "A Tale of Two Comprehensions? Analyzing Student Programmer Attention during Code Summarization". English. In: *ACM Transactions on Software Engineering and Methodology* 33.7 (2024). Publisher: Association for Computing Machinery. ISSN: 15577392 (ISSN); 1049331X (ISSN). DOI: 10.1145/3664808. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85206219352&doi=10.1145%2F3664808&partnerID=40&md5=b9c35caf11138ea0070aa8dfee7c01fb.

[52] N. Mansoor et al. "Assessing the Effect of Programming Language and Task Type on Eye Movements of Computer Science Students". English. In: *ACM Transactions on Computing Education* 24.1 (2024). Publisher: Association for Computing Machinery. ISSN: 19466226 (ISSN). DOI: 10.1145/3632530. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85190970952&doi=10.1145%2F3632530&partnerID=40&md5=699835f49b5d3f8dc39032ad2a82de7f.

[53] M. Andrzejewska and P. Kotoniak. "Development of Program Comprehension Skills by Novice Programmers – Longitudinal Eye Tracking Studies". English. In: *Informatics in Education* 19.4 (2020). Publisher: Vilnius University Institute of Data Science and Digital Technologies, pp. 521–541. ISSN: 16485831 (ISSN). DOI: 10.15388/INFEDU.2020.23. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85099189511&doi=10.15388%2FINFEDU.2020.23&partnerID=40&md5=1f60c64deda30b467adc97df8660e7f3.

[54] S.A. Jessup et al. "Using eye-tracking data to compare differences in code comprehension and code perceptions between expert and novice programmers". English. In: ed. by T.X. Bui. Vol. 2020-January. IEEE Computer Society, 2021, pp. 114–123. ISBN: 15301605 (ISSN); 9780769530758 (ISBN); 0818673249 (ISBN); 0769514359 (ISBN); 0818632305 (ISBN); 9780998133119 (ISBN); 0769530753 (ISBN); 0769525075 (ISBN); 9780998133140 (ISBN); 9780998133126 (ISBN); 9780769525075 (ISBN). URL: https://www.scopus.

com/inward/record.uri?eid=2-s2.0-85108313306&partnerID=40&md5=65a80e4d939c50a52c014416590fa7e8.

[55] K.-I. Park et al. "An eye tracking study assessing source code readability rules for program comprehension". English. In: *Empirical Software Engineering* 29.6 (2024). Publisher: Springer. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-024-10532-x. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85205987420&doi=10.1007%2Fs10664-024-10532-x&partnerID=40&md5=f7d8c87b28f88177c51decac941d7b3d.

[56] K. Mangaroska et al. "Exploring students' cognitive and affective states during problem solving through multimodal data: Lessons learned from a programming activity". English. In: *Journal of Computer Assisted Learning* 38.1 (2022). Publisher: John Wiley and Sons Inc, pp. 40–59. ISSN: 02664909 (ISSN); 13652729 (ISSN). DOI: 10.1111/jcal.12590. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85114298355&doi=10.1111%2Fjcal.12590&partnerID=40&md5=85aa5bfc580ab2ff111d0d588443d558.

[57] S. Fakhoury et al. "Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization". English. In: *Empirical Software Engineering* 25.3 (2020). Publisher: Springer, pp. 2140–2178. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-019-09751-4. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85070284963&doi=10.1007%2Fs10664-019-09751-4&partnerID=40&md5=f1b4c5bc35763b68f6e7d781052ad9e7.

[58] J.A. Saddler et al. "Studying Developer Reading Behavior on Stack Overflow during API Summarization Tasks". English. In: ed. by K. Kontogiannis et al. Institute of Electrical and Electronics Engineers Inc., 2020, pp. 195–205. ISBN: 9781728151434 (ISBN). DOI: 10.1109/SANER48275.2020.9054848. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85083567063&doi=10.1109%2FSANER48275.2020.9054848&partnerID=40&md5=93fb7f16471491db7f63b6ef6224f30e.

[59] J.A. Saddler et al. "Reading Behavior and Comprehension of C++ Source Code - A Classroom Study". English. In: ed. by D.D. Schmorrow and C.M. Fidopiastis. Vol. 11580 LNAI. Springer Verlag service@springer.de, 2019, pp. 597–616. ISBN: 16113349 (ISSN); 03029743 (ISSN); 9789819698936 (ISBN); 9789819698042 (ISBN); 9789819698110 (ISBN); 9789819698905 (ISBN); 9789819512324 (ISBN); 9783032026019 (ISBN); 9783032008909 (ISBN); 9783031915802 (ISBN); 9789819698141 (ISBN); 9783031984136 (ISBN). DOI: 10.1007/978-3-030-22419-6_43. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85069533899&doi=10.1007%2F978-3-030-22419-6_43&partnerID=40&md5=95901c857d07ce3f7ef05d460e575aa7.

[60] P. Kather, R. Duran, and J. Vahrenhold. "Through (Tracking) Their Eyes: Abstraction and Complexity in Program Comprehension". English. In: *ACM Transactions on Computing Education* 22.2 (2022). Publisher: Association for Computing Machinery. ISSN: 19466226 (ISSN). DOI: 10.1145/3480171. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85127197118&doi=10.1145%2F3480171&partnerID=40&md5=91a909ed468d3eeeaff93d45

[61] B. Clark and B. Sharif. "ITraceVis: Visualizing Eye Movement Data Within Eclipse". English. In: vol. 2017-October. Institute of Electrical and Electronics Engineers Inc., 2017, pp. 22–32. ISBN: 9781538610039 (ISBN). DOI: 10.1109/VISSOFT.2017.30. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85047202984&doi=10.1109%2FVISSOFT.2017.30&partnerID=40&md5=0824cf11e26eb0d4a9e26701fc1476d2.

[62] J.A.S. da Costa et al. "Seeing confusion through a new lens: on the impact of atoms of confusion on novices' code comprehension". English. In: *Empirical Software Engineering* 28.4 (2023). Publisher: Springer. ISSN: 15737616 (ISSN); 13823256 (ISSN). DOI: 10.1007/s10664-023-10311-0. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85159964496&doi=10.1007%2Fs10664-023-10311-0&partnerID=40&md5=3477fce96f542edf397a036de3d464

[63] "Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research". en. In: *Advances in Psychology*. Vol. 52. Elsevier, 1988, pp. 139–183. ISBN: 978-0-444-70388-0. DOI: 10.1016/S0166-4115(08)62386-9. URL: https://linkinghub.elsevier.com/retrieve/pii/S0166411508623869 (visited on 11/25/2025).

[64] C.L.S. Tablatin. "Visual Attention Patterns in Finding Source Code Defects". English. In: *WSEAS Transactions on Information Science and Applications* 20 (2023). Publisher: World Scientific and Engineering Academy and Society, pp. 375–389. ISSN: 22243402 (ISSN); 17900832 (ISSN). DOI: 10.37394/23209.2023.20.40. URL: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85178282299&doi=10.37394%2F23209.2023.20.40&partnerID=40&md5=66efcfe353b667360b7b20c8f6cb3867.

# APPENDICES

# A - OVERVIEW OF CODED STUDIES

This appendix provides a detailed overview of all included studies and their corresponding coded values. Research questions, objectives, and limitations are excluded from the table since they are just copied and pasted from the studies and take up a lot of space in the table.

**Table A.1:** Coded overview of all included studies.

| Title | Year | Task | Lang. | Pop. | N | Part. tracked | Unit | Eye Tracker | Eye Metrics | Perf. | Analysis | Feedback (Provided / Type / Trigger) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A Comparative Study of High and Low Performing Students' Visual Effort and Attention When Identifying Syntax Errors [24] | 2024 | Find the error | Java, C++ | University students | 45 | 1 | Individual | Gazepoint | aoi | Effect | inf | No |
| A Tale of Two Comprehensions? Analyzing Student Programmer Attention during Code Summarization [51] | 2024 | Comprehension | Java | Undergraduate, Graduate | 27 | 1 | Individual | Tobii Pro Fusion | fix, seq | Subj | inf, seq | No |
| An Eye Movement Study in Unconventional Usage of Different Software Tools [26] | 2023 | Comprehension | C#, XAML | University students | 36 | 1 | Individual | Gazepoint GP3 | fix, pup | Effect, Subj | inf, des | No |
| An eye tracking study assessing source code readability rules for program comprehension [55] | 2024 | Comprehension | Java | Undergraduate, Graduate, Professional developers | 46 | 1 | Individual | Tobii X60 | fix | Effect, Time, Subj | inf, des | No |
| Analyzing reading pattern of simple c source code consisting of only assignment and arithmetic operations based on data dependency relationship by using eye movement [44] | 2018 | Comprehension | C | Undergraduate, Graduate | 16 | 1 | Individual | Tobii X2-30 | fix, sac | Other | seq, inf | No |
| Analyzing the influence of block highlighting on beginning programmers' reading behavior using eye tracking [49] | 2020 | Comprehension | Java | Novice programmers | 7 | 1 | Individual | Tobii T60 XL | fix | Effect | vis, qual | No |
| Assessing code readability in Python programming courses using eye-tracking [46] | 2024 | Comprehension | Python | Undergraduate, Graduate | 76 | 1 | Individual | GazePoint GP3 | fix, aoi | Subj, Time | corr, ml | No |
| Assessing the Effect of Programming Language and Task Type on Eye Movements of Computer Science Students [52] | 2024 | Programming | C++, Python | Undergraduate, Graduate, Professional developers | 14 | 1 | Individual | Tobii X60 | fix, aoi, seq | Effect, Time | inf, vis | No |
| Atoms of Confusion: The Eyes Do Not Lie [33] | 2020 | Comprehension | C | Undergraduate, Graduate, PhD | 30 | 1 | Individual | N/A | vis, fix, sac | Time, Effect | inf, vis | No |

*Continued on next page*

| Title | Year | Task | Lang. | Pop. | N | Part. tracked | Unit | Eye Tracker | Eye Metrics | Perf. | Analysis | Feedback (Provided / Type / Trigger) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Carry-forward effect: providing proactive scaffolding to learning processes [18] | 2025 | Debugging | Java | Elementary School, Undergraduate, Professional Developers | 166 | 1, 1, 2 | Individual, Individual, Pair | Tobii glasses, Tobii X3-120 | pup, sac, seq | Effect | inf | Yes (On-screen guidance / proactive / CFE rules on MMD signals) |
| Classification of strategies for solving programming problems using AoI sequence analysis [11] | 2019 | Programming, Comprehension | None | Undergraduate | 36 | 1 | Individual | Gazepoint GP3 | fix | Effect, Time | inf, seq, vis | No |
| Coding activities for children: Coupling eye-tracking with qualitative data to investigate gender differences [27] | 2020 | Programming | Scratch | K-12 | 149 | 5 | Individual | 4x SMI, Tobii | fix, sac, aoi | Effect | inf, qual | No |
| Collaborative eye tracking based code review through real-time shared gaze visualization [21] | 2022 | Code review, find the bug | C++ | Undergraduate, College Students | 34 | 2 | Pair | Custom head-mounted eye trackers | aoi, seq | Time | inf, qual | No |
| Developer Reading Behavior while Summarizing Java Methods: Size and Context Matters [35] | 2019 | Comprehension | Java | Undergraduate, graduate, Professional developers, PhD | 18 | 1 | Individual | Tobii X60 | fix | Other | inf, corr | No |
| Development of Program Comprehension Skills by Novice Programmers – Longitudinal Eye Tracking Studies [53] | 2020 | Comprehension | C | Undergraduate | 35 | 1 | Individual | iViewX HiSpeed | fix, sac, seq | Effect, Time, Subj | inf | No |
| Evaluating refactorings for disciplining #ifdef annotations: An eye tracking study with novices [31] | 2021 | Comprehension | C | Undergraduate, Graduate | 64 | Not stated | Individual | N/A | fix | Effect, Time | inf | Yes (sound indicating correctness / reactive / answer submission) |
| Evaluating the Code Comprehension of Novices with Eye Tracking [48] | 2023 | Comprehension | Java, C, Python | Undergraduate | 128 | 1 | Individual | N/A | fix | Time, Effect | des, qual | No |
| Expertise-dependent visual attention strategies develop over time during debugging with multiple code representations [50] | 2012 | Comprehension | Java | Graduate, Professional developers | 14 | 1 | Individual | Tobii 1750 | fix, sac | Effect, Time | seq, inf, corr | No |
| Exploring differences in self-regulated learning strategy use between high- and low-performing students in introductory programming: An analysis of eye-tracking and retrospective think-aloud data from program comprehension [42] | 2024 | Comprehension | Python | Undergraduate | 62 | 1 | Individual | Gazepoint GP3 | fix | Other | inf, des | No |

*Continued on next page*

| Title | Year | Task | Lang. | Pop. | N | Part. tracked | Unit | Eye Tracker | Eye Metrics | Perf. | Analysis | Feedback (Provided / Type / Trigger) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Exploring students' cognitive and affective states during problem solving through multimodal data: Lessons learned from a programming activity [56] | 2022 | Find the bug | Java | University students | 40 | 2 | Individual | Tobii X3-120 | pup, fix, sac, aoi | Effect | reg, corr, inf, vis | No |
| Exploring the Effects of Urgency and Reputation in Code Review An Eye-Tracking Study [22] | 2024 | Code review | Java | Undergraduate, Graduate | 37 | 1 | Individual | Tobii Pro X3-120 | fix, sac, aoi, pup | Effect, Time, Subj | inf, corr, des | No |
| From Novice to Expert: Analysis of Token Level Effects in a Longitudinal Eye Tracking Study [41] | 2021 | Comprehension | Java | Novice programmers, Professional developers | 20 | 1 | Individual | SMI RED-m remote | fix | Other | inf, ml | No |
| Gaze collaboration patterns of successful and unsuccessful programming pairs using cross-recurrence quantification analysis [20] | 2019 | Debugging | Java | Undergraduate | 84 | 2 | Pair | Gazepoint GP3 | seq | Effect | seq, inf | No |
| How programmers read regular code: a controlled experiment using eye tracking [38] | 2017 | Comprehension | C | Undergraduate, Faculty members | 20 | 1 | Individual | Eye Tribe tracker | fix | Effect, Time, Effort | inf | No |
| Identifying Code Reading Strategies in Debugging using STA with a Tolerance Algorithm [36] | 2022 | Find the bug | Java | College students | 63 | 1 | Individual | Gazepoint GP3 | fix, aoi | Effect | inf, corr, seq | No |
| ITraceVis: Visualizing Eye Movement Data Within Eclipse [61] | 2017 | Comprehension | Java | Graduate | 10 | 1 | Individual | Tobii X3-120 | fix, pup | Effect, Subj | des, qual | No |
| Looking "Under the hood" of learning computer programming: the emotional and cognitive differences between novices and beginners [28] | 2024 | Programming | Scratch | Undergraduate | 23 | 1 | Individual | Tobii X60 | pup | Other | reg, inf | Yes (Simulation of the program / Reactive / after running the code) |
| Looking at the main Method - An Educator's Perspective [47] | 2021 | Comprehension | Java | Novice programmers, Professional developers | 26 | 1 | Individual | SMI RED-m remote | aoi, fix | Effect | inf | No |
| Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization [57] | 2020 | Comprehension, find the bug | Java | Undergraduate, Graduate | 25 | 1 | Individual | Eye Tribe tracker | fix | Effect, Time | corr | No |
| Predicting expertise among novice programmers with prior knowledge on programming tasks [45] | 2020 | Comprehension | Java | Undergraduate | 60 | 1 | Individual | Tobii X2-30C | fix, aoi | Effect | ml, inf | No |

*Continued on next page*

| Title | Year | Task | Lang. | Pop. | N | Part. tracked | Unit | Eye Tracker | Eye Metrics | Perf. | Analysis | Feedback (Provided / Type / Trigger) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Predicting Pair Success in a Pair Programming Eye Tracking Experiment Using Cross-Recurrence Quantification Analysis [37] | 2022 | Find the bug | Java | College students | 84 | 2 | Pair | Gazepoint | seq | Effect | corr, ml | No |
| Reading Behavior and Comprehension of C++ Source Code - A Classroom Study [59] | 2019 | Comprehension | C++ | Undergraduate, Graduate | 15 | 1 | Individual | Tobii X60 | fix, sac, seq | Effect | inf, vis | No |
| Reading skills can predict the-programming performance of novices: An eye-Tracking study [39] | 2019 | Comprehension | C++ | Undergraduate | 16 | 1 | Individual | Tobii T120 | vis, seq | Effect | vis, qual | No |
| Seeing confusion through a new lens: on the impact of atoms of confusion on novices' code comprehension [62] | 2023 | Comprehension | Python | Undergraduate | 32 | 1 | Individual | Tobii Eye Tracker 4C | fix, seq, aoi | Time, Effect | inf | Yes (correctness indicator / reactive / answer submission) |
| Simultaneous measurement of program comprehension with fMRI and eye tracking: A case study [23] | 2018 | Comprehension, find the error | Java | Undergraduate, Graduate | 20 | 1 | Individual | Eyelink 1000 | fix | Effect | vis, inf | No |
| Studying Developer Eye Movements to Measure Cognitive Workload and Visual Effort for Expertise Assessment [29] | 2023 | Comprehension | Java | Novice programmers, Professional developers | 207 | 1 | Individual | SMI RED 250 remote | fix, sac, pup, aoi | Effect | corr, vis | No |
| Studying Developer Reading Behavior on Stack Overflow during API Summarization Tasks [58] | 2020 | Comprehension | Java | Graduate, Professional developers, | 30 | 1 | Individual | Tobii X60 | fix, seq | Effect | inf | No |
| Syntax highlighting as an influencing factor when reading and comprehending source code [25] | 2016 | Comprehension | C# | Undergraduate, Graduate | 34 | 1 | Individual | Tobii TX-300 | fix | Effect, Subj | inf, oth | No |
| The impact of identifier style on effort and comprehension [30] | 2013 | Comprehension | Java, C | Undergraduate, Graduate, Faculty members, Non-programmers, University students, Professional developers | 169 | 1 | Individual | Tobii 1750 | fix, sac | Effect, Time, Effort | reg, inf | No |

*Continued on next page*

| Title | Year | Task | Lang. | Pop. | N | Part. tracked | Unit | Eye Tracker | Eye Metrics | Perf. | Analysis | Feedback (Provided / Type / Trigger) |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers [40] | 2024 | Programming | C++ | Undergraduate | 20 | 1 | Individual | Tobii | fix, sac, vis | Time, Effect | qual, vis | Yes (Automated Assessment Tool (Athene) test-case results, AI suggestions, error feedback / reactive / submission) |
| Through (Tracking) Their Eyes: Abstraction and Complexity in Program Comprehension [60] | 2022 | Comprehension | Java | Graduate, Professional developers | 17 | 1 | Individual | Tobii Pro Spectrum | aoi, fix, sac | Effect, Time | inf, qual | No |
| Tracing software developers' eyes and interactions for change tasks [19] | 2015 | Debugging | Java | Professional developers, Undergraduate, Graduate | 22 | 1 | Individual | Tobii X60 | aoi, fix, pup | Effect, Effort | des, inf, seq | No |
| Tracking Students' Cognitive Processes during Program Debugging-An Eye-Movement Approach [5] | 2016 | Debugging | C | Undergraduate | 38 | 1 | Individual | Eyelink 1000 | fix, seq | Effect | inf, seq | No |
| Using developer eye movements to externalize the mental model used in code summarization tasks [34] | 2019 | Comprehension | Java | Undergraduate, PhD, Professional developers | 18 | 1 | Individual | Tobii X60 | fix | Other | inf, seq | No |
| Using eye-tracking data to compare differences in code comprehension and code perceptions between expert and novice programmers [54] | 2021 | Comprehension | Java | Novice programmers, Professional developers | 20 | 1 | Individual | Smart Eye Aurora remote | fix | Effect | reg, inf | No |
| Visual Attention Patterns in Finding Source Code Defects [64] | 2023 | Find the bug | Java | Undergraduate | 63 | 1 | Individual | Gazepoint | fix, aoi | Effect | inf | No |
| Visually analyzing students' gaze on C++ code snippets [43] | 2019 | Comprehension | C++ | Undergraduate, Graduate | 15 | 1 | Individual | Tobii X60 | fix, sac, aoi | Other | vis, seq | No |
| What drives the reading order of programmers? an eye tracking study [32] | 2020 | Comprehension | Java | Graduate, PhD, Undergraduate | 31 | 1 | Individual | Tobii EyeX | seq | Effect, Time | reg, inf | No |

# B - FIGMA PROTOTYPE

Below, initial prototypes are attached to present a possible user interface.

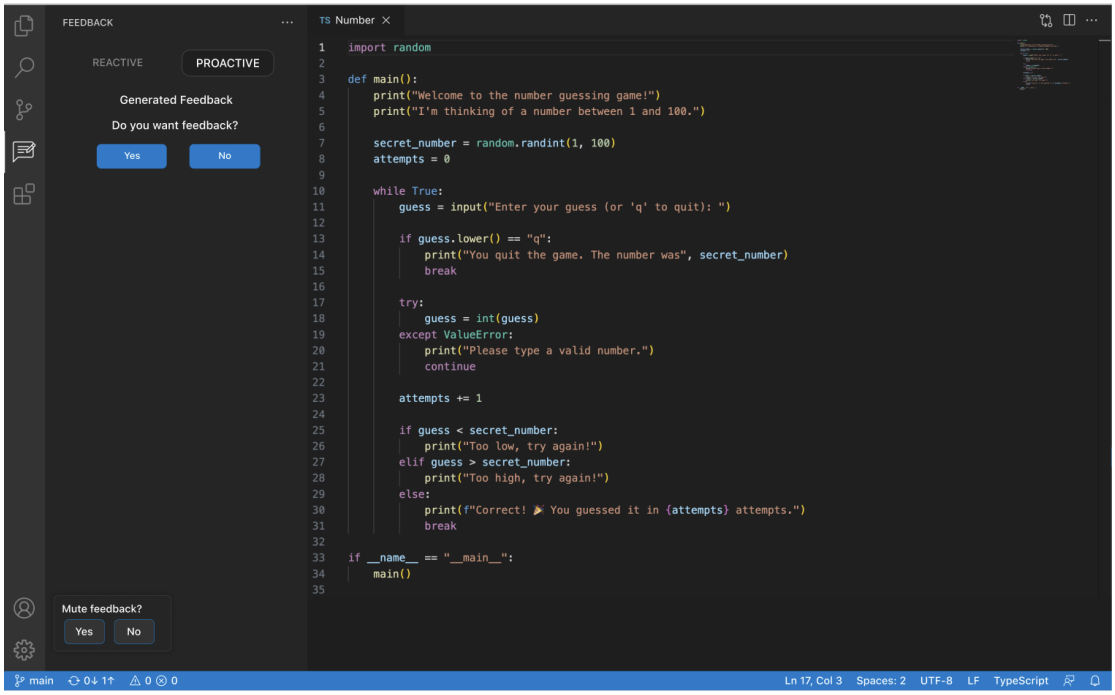



**Figure B.1:** Example of UI after feedback is generated

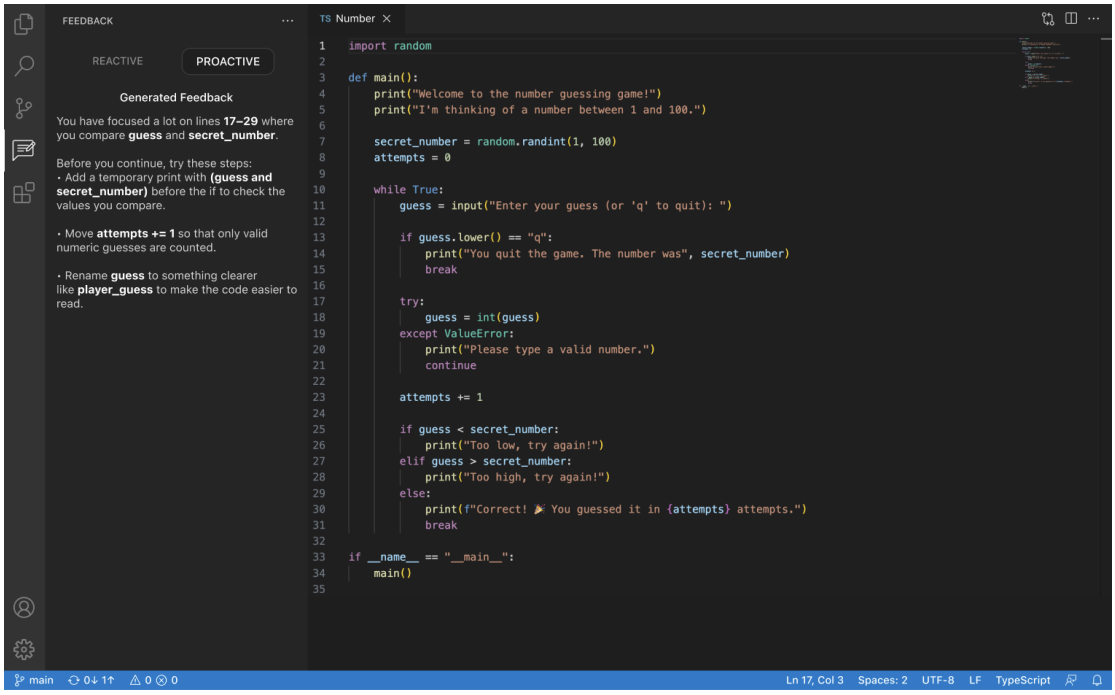



**Figure B.2:** Example of UI after feedback is presented

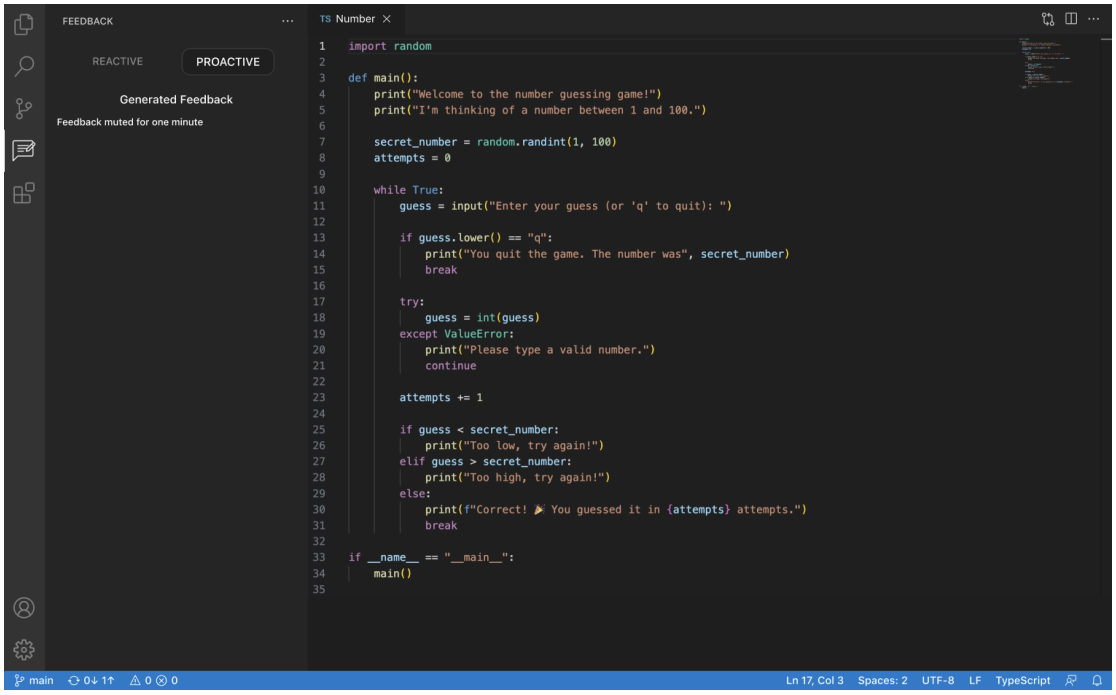



**Figure B.3:** Example of UI after feedback is rejected

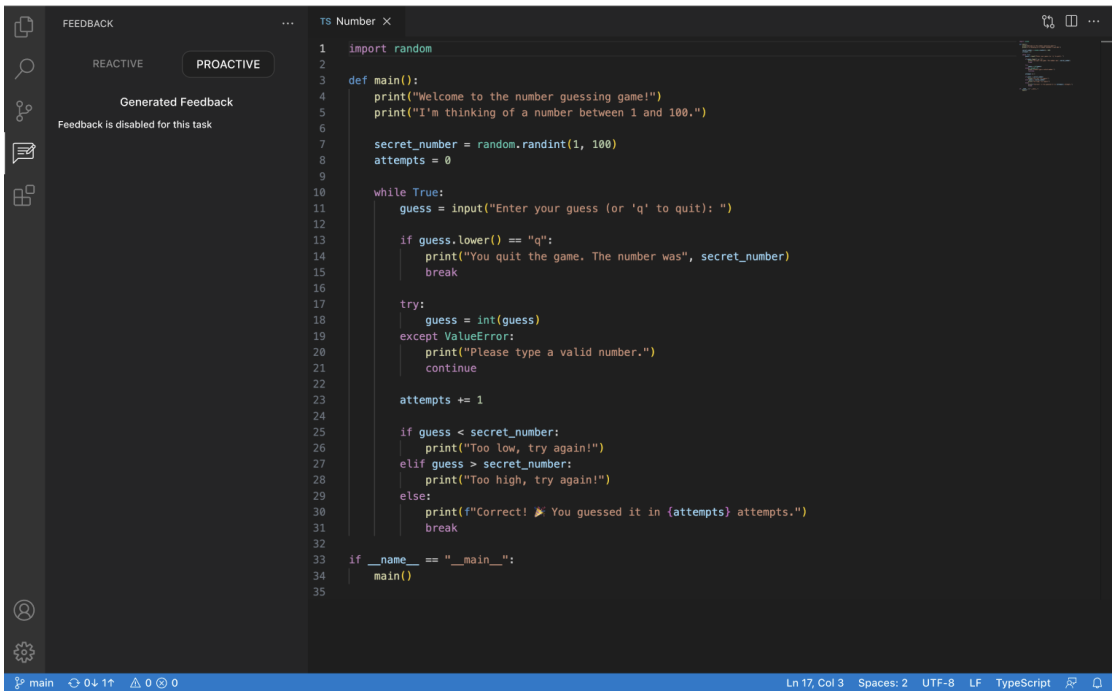



**Figure B.4:** Example of UI after feedback is disabled